一、Hello的区别
1、文件扩展名不同
2、头文件不同
3、编译器不同
4、输入、输出不同
5、名字空间
6、注意
二、基本数据类型的不同
1、bool类型
2、void类型的指针
3、字符串
三、结构、联合、枚举的区别
1、结构、联合的区别
2、枚举的区别
四、函数的区别
1、C++的函数可以重载
构成函数重载的条件：
函数重载的原理：
C++中如何使用C语言的库文件：
面临的问题：
解决方法：
2、默认形参
3、内联函数
编译器优化：
与宏函数的相同点和不同点：
相同点：
不同点：
什么样的函数适合设置为内联函数：
五、堆内存管理的区别
C语言的内存管理：
C++的堆内存管理：
new/delete和malloc/free的相同点：
问题：delete[] 释放内存时为什么可以调用n次析构函数。
问题：在C++中如何把已有一块内存分配给结构、联合、类对象，能自动调用构造、析构函数。
六、引用和指针
1、什么是引用
2、为什么使用引用
3、使用引用要注意的问题
4、总结
七、类型转换的区别
隐式类型转换：
强制类型转换：
八、名字空间
1、什么是名字空间
2、如何设计名字空间
3、如何使用名字空间中的内容
方法1：
方法2：
方法3：
4、空间合并与空间嵌套
5、匿名名字空间
6、注意
九、面向过程与面向对象

1、什么是面向过程编程
2、什么是面向对象编程
抽象：
封装：
继承：
多态：
十、类和对象
1、什么是类
2、如何设计类
3、访问限制符
4、什么是类对象
5、类的声明与实现分开
十一、构造函数与析构函数
构造函数
1、什么构造函数
2、何时调用构造函数
3、在构造函数中应该做什么
4、实现构造函数要注意的问题
5、explicit关键字的作用
析构函数：
1、什么析构函数
2、在析构函数中应该做什么
3、何时调用析构函数
4、析构函数要注意的问题
十二、初始化列表与const成员
1、const成员
2、初始化列表
十三、this指针与常函数
1、成员函数是如何区别调用它的对象？
C语言中我们如何解决该问题：
在C++语言中该问题由编译器帮忙解决：
2、常对象与常函数
什么是常函数
常函数的特点：
什么是常对象
常对象的特点：
常对象有常函数的局限性：
3、mutable关键字的作用
4、空类的对象为什么占用1字节内存？
十四、拷贝构造函数和赋值函数
1、什么是拷贝构造
2、什么会调用拷贝构造
3、拷贝构造的任务是什么
4、什么时候需要显式实现拷贝构造
5、什么是赋值函数
6、什么时候会调用赋值函数
7、赋值函数的任务是什么
8、什么时候需要显式实现赋值函数
9、浅拷贝与深拷贝
10、自定义MyString类
十五、静态成员
1、普通成员
普通成员变量的特点：
普通成员函数的特点：
2、静态成员
什么是静态成员变量：
静态成员变量的特点和局限性：
什么是静态成员函数：

静态成员函数的特点和局限性：
静态成员的作用
十六、单例模式
1、什么是单例模式
2、什么时候需要使用单例模式
3、单例模式的实现原理
4、饿汉单例模式
5、懒汉单例模式
6、线程安全的懒汉单例
十七、运算符函数
1、什么是运算符函数
2、全局运算符函数
3、成员运算符函数
十八、运算符重载
1、双目运算符重载
重载为成员运算符函数：
重载为全局运算符函数：
友元函数：
2、输入、输出运算符重载
3、单目运算符的重载
重载为成员运算符函数：
重载为全局运算符函数：
4、自变运算符重载
前自变运算符重载 ++i/--i：
后自变运算符重载 ++i/--i：
5、new/delete运算符的重载
new运算符的重载：
格式1：new TYPE
格式2：new(ptr) TYPE
new/delete运算符有什么用：
十九、智能指针
1、常规指针的缺点
2、智能指针的优点
3、智能指针与常规指针的相同点
4、智能指针与常规指针的不同点
5、简化版的智能指针类
6、C++89标准中的智能指针auto_ptr
如何使用auto_ptr：
auto_ptr的基本用法：
auto_ptr的局限性：
7、C++11中的新版智能指针
unique_ptr型智能指针：
shared_ptr共享型智能指针：
weak_ptr弱引用型智能指针：
二十、运算符重载的局限性
1、无法重载的运算符
2、单目运算符只适合重载为成员函数
3、重载运算符时要注意的问题
二一、类继承
1、什么是类继承
2、类继承的语法格式
3、继承方式对访问权限的影响
成员的访问权限：
继承方式对成员的影响：
4、C++语言的继承特点
二二、对象的创建与释放过程
1、创建对象的过程
2、释放对象的过程

二三、钻石继承与虚继承
1、什么是钻石继承
2、钻石继承危害
3、什么是虚继承
4、虚继承的原理
二四、虚函数与函数覆盖(重写)
1、什么虚函数
2、什么是函数覆盖
3、构成函数覆盖的条件
4、函数覆盖的原理
二五、多态
1、什么是多态
2、编译时多态（静态多态）
3、运行时多态（动态多态、类多态）
4、构成类多态的原理
5、类多态的优点
二六、纯虚函数和抽象类
1、什么是纯虚函数
2、什么是抽象类
3、什么是纯抽象类
二七、虚构造与虚析构
1、构造函数能否是虚函数，为什么？
2、析构函数能否是虚函数，为什么？
3、什么进修需要设计虚析构
二八、类型信息运算符
1、什么类型信息运算符
2、类型信息运算符的作用。
3、使用方法
二九、强制类型转换
1、去常类型转换 const_cast<目标类型>(源数据)
2、静态类型转换 static_cast<目标类型>(源数据)
3、重解释类型转换 reinterpret_cast<目标类型>(数据)
4、动态类型转换 dynamic_cast<目标类型> (数据)
三十、C++的文件IO
1、打开文件操作
使用构造函数打开文件：
使用open成员函数打开文件：
如何判断文件打开成功或失败：
方法1：直接使用类对象进行逻辑判断，因它们重载逻辑运算符。
方法2：调用good函数，该函数用于判断对象的上一次操作是否成功，所以也可判断文件打开是否成功。
2、文本格式读写
3、二进入格式读写
4、随机读写
5、特殊格式的读写
三一、 异常处理
1、什么是异常处理
2、C语言的异常处理
3、C++语言的异常处理
如何抛异常：
声明异常：
捕获异常：
抛异常和捕获异常要注意的问题：
4、C++标准异常
5、自定义的通用异常

一、Hello的区别

```
#include <iostream>
using namespace std;

int main(int argc,const char* argv[])
{
    cout << "Hello World!" << endl;
    return 0;
}
```

1、文件扩展名不同

cpp、C、cxx

2、头文件不同

C++语言的标准库文件，文件名的末尾不带.h，iostream用于标准输入输出头文件，C语言的相关头文件还可以继续使用。

为了统一命名风格，C++为C语言重定义了不带.h标准库头文件，例如：stdio.h重定义了cstdio。

自定义的头文件，还可以继续以.h结尾。

3、编译器不同

g++，相关参数的使用方法与gcc一样。

4、输入、输出不同

cout、cin是用于输入、输出的标准库类对象。

cout << 要输出的数据 << endl，多个数据用<<隔开。

cin >> 变量名，多个数据使用>>隔开。

cout和cin可以自动识别数据类型，但输入输出复杂格式的数据时，没有printf和scanf好用。

<< 和 >> 还可以进行运算符重载，但左移、右移的功能依然保留。

printf和scanf还可以继续使用，但需要包含相关的头文件。

5、名字空间

为了避免命名冲突，C++中引入了一项命名空间的管理技术 **名字空间**，后续再讲解。

6、注意

C++基本上完全兼容C语言的所有内容。

二、基本数据类型的不同

1、bool类型

在C++中，bool就是一种真正的基本数据类型，不需要包含头文件了，bool、true、false是C++中的关键字。

可以给bool类型的变量赋值整数，但它会自动转换成0或1。

```cpp
#include <iostream>
using namespace std;

int main(int argc,const char* argv[])
{
    bool flag;
    cout << sizeof(bool) << " " << sizeof(true) << " " << sizeof(false) << endl;

    flag = 4;
    cout << flag << endl; // 输出的1，会把整数自动转换成0|1。
    return 0;
}
```

2、void类型的指针

在C语言void类型的指针可以与其它类型的指针自动转换，而在C++中：

其它类型指针可以自动转换成 void*，之所以保留是因为C标准库、操作系统、第三方库中有大量的函数的参数使用了void*作为参数，如果该功能不保留，这类函数就无法再正常调用。

void* 不能再自动转换成 其它类型指针，为了安全C++语言对类型检查比C语言要严格很多。

```cpp
#include <iostream>
#include <cstdlib>
using namespace std;

int main(int argc,const char* argv[])
{
    int* p = (int*)malloc(40);
    return 0;
}
```

3、字符串

在C语言中使用char类型的数组或char*指针指向的内存来存储字符串，使用string.h中的函数操作字符串，但在C++中使用string类型字符串变量，使用相关的运算符操作字符串。

```cpp
#include <iostream>
using namespace std;

int main()
{
    // 定义字符串对象
    string str;
    // 输入字符串，不用关心存储空间是否够用，会自动扩展
    cin >> str;
    // 输出字符串
    cout << str << endl;

    // 给字符串赋值，strcpy
    str = "hello";
    // 追加字符串，strcat
    str += "world";
    // 计算字符串长度，strlen
    cout << str.size() << endl;
```

```
    // 比较字符串，== != > < >= <=
    cout << (str == "xixi") << endl;
}
```

注意：string类型的底层，依然是使用char类型的指针、数组实现的，并不是一种全新的类型，而是对char字符串的封装。

三、结构、联合、枚举的区别

1、结构、联合的区别

1、在C++中定义结构、联合对象时，struct、union关键字可以省略。

2、在C++中结构、联合的内部，可以有成员函数，使用结构、联合对象加.或->调用，成员函数的内部可以直接使用成员变量，成员函数可以自动区别对象的成员变量。

3、在C++中结构、联合中可以对成员变量、成员函数进行访问权限的管理：

private　私有的成员

public 公开的成员

protected 受保护的成员

4、在C++中创建、销毁结构、联合对象时，会自动调用构造函数(以结构名命名）、析构函数（~结构名命名）。

```
#include <iostream>
using namespace std;

struct Student
{
    int id;
    char name[20];
    short age;
    void show(void)
    {
        cout << id << " " << name << " " << age << endl;
    }
    Student(void)
    {
        cout << "我是构造函数" << endl;
    }
    ~Student(void)
    {
        cout << "我是析构函数" << endl;
    }
};

union Data
{
    char ch;
    int num;
};

int main(int argc,const char* argv[])
{
```

```
    /*
    Student stu1 = {10010,"hehe",28};
    Student stu2 = {10011,"xixi",30};
    stu1.show();
    stu2.show();
    */
    Student stu;
    Data d;

    return 0;
}
```

2、枚举的区别

1、定义枚举变量时，enum关键字可以省略。

2、C++中的枚举不再是int类型模拟的，整数不能给枚举变量赋值。

```
#include <iostream>
using namespace std;

enum DirectionKey
{
    Up,Down,Right,Left
};

int main(int argc,const char* argv[])
{
    DirectionKey key;
    // key = 1234;
    key = Down;
    cout << key << endl;
    return 0;
}
```

四、函数的区别

1、C++的函数可以重载

1、C++中的函数重名，我们把这项技术叫函数重载。

2、重载的函数，参数列必须不同，即参数的个、类型不同。

3、调用函数时，如果与实参相符的函数没有定义，则可以对实参进行自动类型转换调用相关函数，如果实参进行自动类型转换后有多个选则会导致调用冲突。

```
#include <iostream>
using namespace std;

void func(long num)
{
    cout << "func long类型参数" << endl;
}

void func(short num)
{
    cout << "func short类型参数" << endl;
```

```
}

int main(int argc,const char* argv[])
{
    func(1234);
    return 0;
}

```

4、如果函数的参数是指针或引用，指针变量是否使用const修饰会影响函数重载。

```
void func(int* p)
{
    cout << "func int* p" << endl;
}

void func(const int* p)
{
    cout << "func const int* p" << endl;
}

int main(int argc,const char* argv[])
{
    int num1;
    func(&num1); // void func(int* p)
    const int num2;
    func(&num2); // void func(const int* p)
    return 0;
}
```

构成函数重载的条件：

1、同一个作用域

2、函数名相同

3、参数列表不同（参数的个数、类型，指针、引用是否加const）

函数重载的原理：

C++中的函数重载并不是真正的重名，而是在编译时编译器会把函数的参数列表信息追加到函数名的末尾，也就是在编译时函数名经历的换名的过程，在函数调用时，编译器会把实参的类型信息追加到函数名的末尾，这样就知道该调用哪个函数了。

```
// _Z4funcPi:
void func(int* p)
{
    cout << "func int* p" << endl;
}

// _Z4funcPKi:
void func(const int* p)
{
    cout << "func const int* p" << endl;
}

// _Z4funcii:
```

```
void func(int num,int num1)
{
    cout << "func const int int" << endl;
}
```

C++中如何使用C语言的库文件：

面临的问题：

1、把函数声明头文件导入后，默认情况下g++会按照C++的语法声明函数，会把C头文件中的函数声明进行换名。

2、g++在编译调用函数的语句时，会先尝试调用换名的函数，此时发现已经声明过，所以编译器生成的换名的函数调用指令，而C的库文件中的函数没有换名，所以就会调用失败，也就链接失败。

解决方法：

1、使用extern "C" 包含一下C函数声明，这个语句的功能就是告诉编译器按照C语言的处理方式编译函数声明，也就是不要对函数进行换名。

2、g++在编译调用函数的语句时，会先尝试调用换名的函数，此时会发现没有该版本的函数声明，然后再尝试调用未换名的函数，这样编译器生成就是未换名的函数调用指令，就能成功调用C的库文件中的函数。

2、默认形参

1、在声明函数时，可以给函数的形式参数设置一个默认值，当调用函数时，设置过默认值的参数位置可以不提供实参，会使用默认值。

```
#include <iostream>
using namespace std;

void func(int num1=1234,int num2)
{
    cout << num << endl;
}

int main(int argc,const char* argv[])
{
    func(123);
    return 0;
}
```

2、设置默认值的参数要连续且靠右，因为调用者提供的实参要优先提供给没有设置默认的参数使用。

```
// 错误写法
void func(int num1,int num2=1234,int num3)
{
    cout << num1 << " " << num2 << " " << num3 << endl;
}

int main(int argc,const char* argv[])
{
    func(1,2,3);
    return 0;
}
```

3、如果函数的声明和定义分开实现，则只能在声明函数时设置默认形参，定义函数时不能设置默认形参，因为没有意义。

```
void func(int num1,int num2,int num3=1234);

int main(int argc,const char* argv[])
{
    func(1,2);
    return 0;
}

void func(int num1,int num2,int num3)
{
    cout << num1 << " " << num2 << " " << num3 << endl;
}
```

函数设置默认形参可能会造成调用时的冲突，如果该函数进行重载，又对部分函数设置的默认形参，就可能导致调用函数时有多个备选方案，造成调用冲突。

```
void func(int num1,int num2)
{
    cout << num1 << " " << num2 << endl;
}

void func(int num1,int num2,int num3=1234)
{
    cout << num1 << " " << num2 << " " << num3 << endl;
}

int main(int argc,const char* argv[])
{
    func(1,2);
    return 0;
}
```

3、内联函数

C++标准委员会设计一种特殊函数，函数在声明时在返回值类型的前面增加 inline 关键字，当调用该函数时，编译器不会生成跳转指令，而是函数的二进制指令直接拷贝到调用位置，这样执行该函数时，直接执行二进制指令，不需要跳转，也不需要返回，所以执行速度比普通函数，就像宏函数，但过多使用会造成冗余，代码段变大。

注意：内联函数只是C++标准委员会设计方案，具体是否内联要看编译厂商是否实现了内联功能。

编译器优化：

gcc/g++ -On 设置编译器的优化级别

默认情况下是-O0，不进行内联，-O2以上才会进行内联。

与宏函数的相同点和不同点：

相同点：

没有跳转、返回过程，会提高代码执行速度，都会造成过多使用会造成冗余，代码段变大。

不同点：

1、内联函数会检查参数的类型和个数，宏函数只会检查参数的个数而不会检查类型，内联函数比宏函数安全。

2、宏函数提供任何类型的参数都可以调用，但内联函数的实参只能使用部分类型，宏函数比内联函数的通用性强。

3、内联函数可以有返回值，而宏函数没有。

什么样的函数适合设置为内联函数：

与宏函数一样，请参考C语言的笔记。

注意：结构、联合、类的成员函数默认都设置为了内联函数，这种内联被称为隐式内联，使用inline修饰的函数被称为显式内联。

五、堆内存管理的区别

C语言的内存管理：

1、C语言中没有堆内存管理的语句，而是在C标准库中提供了一套堆内存管理的函数，在使用时还需要包含stdlib.h头文件。

2、malloc系列函数申请堆内存时需要提供字节数，可能会出现字节数不够或过多的情况。

3、malloc系列函数void类型的地址，如果想在C++中继续使用malloc系列函数，则必须对返回值进行强制类型转换，原因就是在C++中void类型的地址不能再自动转换成其它类型的地址。

4、malloc系列函数不能对申请到的内存设置初值，只有calloc函数可以把申请到的内存初始化0。

5、在C++中malloc系列函数为结构、联合、类对象申请、释放内存时，不会自动调用构造、析构函数。

6、在申请数组型的内存块时，可以使用malloc或calloc，释放时也使用free函数。

7、malloc系列函数申请内存失败时会返回NULL地址。

C++的堆内存管理：

1、C++中有堆内存管理的语句，可以new、delete运算符管理堆内存，直接使用不需要包含任何对文件。

2、new运算符在申请内存时只需要提供数据类型，它会自动计算所需要的字节数，每次申请的字节数都刚刚好，不多不少。

3、new运算符返回的是类型的地址，申请时提供是什么类型，返回的地址就是什么类型的。

4、new在申请内存时，还可以对申请到的内存设置初值。

5、new/delete为结构、联合、类对象申请、释放内存时，会自动调用构造、析构函数。

6、在申请数组型的内存块时，使用new 类型[n]，释放内存时使用delete[]，不能与new/delete不能混用，因为new[]/delete[]会自动调用n次构造、析构函数，而new/delete只调用一次构造、析构函数。

7、当使用new申请内存失败时，会抛出std::bad_alloc异常，而不是返回空地址。

```
#include <iostream>
```

```
#include <stdlib.h>
using namespace std;

struct Student
{
    int id;
    char name[20];
    short age;
    float score;
    Student(void)
    {
        cout << "构造函数" << endl;
    }
    ~Student(void)
    {
        cout << "析构函数" << endl;
    }
};

int main(int argc,const char* argv[])
{
    int* p = new int(123456);
    cout << *p << endl;
    delete p;

    /*
    Student* stup = new Student;
    delete stup;
    */
    /*
    Student* stup = (Student*)malloc(sizeof(Student));
    free(stup);
    */
    /*
    Student* stus = new Student[~0];
    delete[] stus;
    */
    return 0;
}
```

new/delete和malloc/free的相同点：

1、都可以管理堆内存。

2、返回的都是地址，只是类型不同。

3、都必须配合指针变量使用。

4、delete和free都可以释放空指针，但都不能重复释放。

问题：delete[] 释放内存时为什么可以调用n次析构函数。

使用new[] 申请数组型的堆内存时(结构、联合、类对象)，所申请内存块的前4个字节，记录着内存块的块数，所以使用delete []释放内存时可以准确调用n次析构函数。

```
    Student* stus = new Student[13];
    cout << *((int*)stus-1) << endl;
    delete[] stus;
```

问题：在C++中如何把已有一块内存分配给结构、联合、类对象，能自动调用构造、析构函数。

```
// new(内存地址) 类型; new运算符可以重新解释一块内存，并自动调用构造函数
int main(int argc,const char* argv[])
{
    void* ptr = malloc(sizeof(Student));
    cout << ptr << endl;
    Student* stup = new(ptr) Student;
    cout << stup << endl;
    delete stup;
    return 0;
}
```

六、引用和指针

1、什么是引用

引用是一种取名机制，它可以给变量重新取一新的名字，所以引用也叫别名。

2、为什么使用引用

1、跨函数共享变量，把函数的参数设置引用，可以在函数内共享实参变量，并且是否共享实参变量由函数的实现者决定，这种设计给了函数实现者权限，并且给了函数调用者方便。

```
#include <iostream>
using namespace std;

void func(int& dashixiong)
{
    dashixiong = 5678;
    cout << &dashixiong << " " << dashixiong << endl;

}

int main(int argc,const char* argv[])
{
    int sunlingling = 1234;
    int& dashixiong = sunlingling;

    func(sunlingling);
    cout << &sunlingling << " " << sunlingling << endl;
    return 0;
}
```

2、提高函数的传参效率，默认情况下，函数传参单向值传递，变量有多少个字节就要拷贝多个字节的数据，而传递变量的地址，可以需要拷贝4字节的数据，而使用引用一个都不需要拷贝，它比指针的传参的效率还要高。

```
#include <iostream>
using namespace std;

struct Data
{
    char str[20];
    int num;
    char ch;
```

```
};

void func(Data& d)
{

}

int main(int argc,const char* argv[])
{
    Data d;
    for(int i=0; i<200000000; i++)
    {
        func(d);
        d.num++;
    }
    return 0;
}
```

3、使用引用要注意的问题

引用在定义时必须初始化，所以不可能存在空引用，但可能存在悬空引用，但使用指针可能空指针、野指针，所以使用引用比使用指针安全。

```
#include <iostream>
using namespace std;

int& func(void)
{
    int num = 1234;
    return num;
}

int main(int argc,const char* argv[])
{
    /*
    int& hehe = func();
    cout << hehe << endl;
    */
    int* p = new int(1234);
    cout << *p << endl;
    int& num = *p;
    delete p;
    cout << num << endl;
    return 0;
}
```

可以引用常量数据，但需要定义const类型的引用。

```
int main(int argc,const char* argv[])
{
    const int num = 1234;
    const int& hehe = num;
    const int& xixi = 56789;

    cout << hehe << endl;
    cout << num << endl;
    cout << xixi << endl;

    return 0;
}
```

函数的参数使用引用时，实参变量就有被修改的风险，为了防止实参变量被破坏，可以使用const修饰引用。

```
void func(const int& num)
{
    num = 23456789;
}

int main(int argc,const char* argv[])
{
    int num = 1234;
    func(num);
    cout << num << endl;

    return 0;
}
```

可以引用一个数组，或定义数组的引用，但不能定义引用型的数组。

```
int main(int argc,const char* argv[])
{
    int arr[5] = {1,2,3,4,5};
    // 可以定义数组指针
    int (*arr1)[5] = &arr;
    // 可以定义指针数组
    int* arr2[5];

    // 可以定义数组引用或引用数组
    int (&arr3)[5] = arr;

    // 不可以定义引用型的数组
    int& arr4[5];

    return 0;
}
```

4、总结

在C++中尽量多使用引用，少使用指针。

七、类型转换的区别

隐式类型转换：

C语言和C++语言隐式类型转换区别不大，仅的区别有：

1、整数数据不能再隐式转换成枚举。

2、void类型的指针不能再隐式转换成其它类型的指针。

强制类型转换：

1、C语言中的强制类型转换语法在C++中还可以继续使用，但官方不建议这样，因为有安全隐患。

2、C++中设计出一套新强制类型转换的规则：

```
reinterpret_cast<目标类型>(数据)
static_cast<目标类型>(数据)
const_cast<目标类型>(数据)
dynamic_cast<目标类型>(数据)
```

3、虽然使用起来比较麻烦，没有C语言方便，但是它能检查程序员转换是否安全，并提示错误。

4、之所以设计这么复杂就是为了让程序员记不住，不使用强制类型转换，因为C++之父本贾尼·斯特劳斯特卢普认为好的代码设计就不应该使用强制类型转换，当需要强制类型转换时，他希望程序员去优化自己的代码不是使用强制类型转换。

八、名字空间

1、什么是名字空间

在C语言中定义的全局变量、函数、结构、联合、枚举、枚举值都在全局作用下，所以当项目比较庞大时，非常可能造成命名冲突，所以C++中选择把全局作用域进行拆分成子作用域进行管理，这些子作用域就是作名字空间。

2、如何设计名字空间

```
namespace 空间名 {
    // 子作用域
    在该作用域中定义全局变量、函数、结构、联合、枚举、枚举值...，不会与全局变量中的命名冲突
} // 此处没有分号
```

3、如何使用名字空间中的内容

方法1：

直接使用空间中的内容

空间名::标识符;

```
#include <iostream>

int main(int argc,const char* argv[])
{
    std::cout << "我要使用std中的内容" << std::endl;
    return 0;
}
```

方法2：

把空间中的部分内容导入到当作用域下，一旦导入之后就可以方便的直接使用

using 空间名::标识符;

```
#include <iostream>

using std::cout;
using std::endl;

int main(int argc,const char* argv[])
{
    cout << "我要使用std中的内容" << endl;
    return 0;
}
```

方法3：

把空间中的所有内容导入到当前作用域下

using namespace 空间名;

```
#include <iostream>

using namespace std;

int main(int argc,const char* argv[])
{
    string str = "string类也定义在std名字空间中";
    cout << str << endl;
    return 0;
}
```

注意：C++标准库中提供的基础功能的类、对象都定义在std名字空间中，如：cout、cin、string。

4、空间合并与空间嵌套

1、编译器会把同名的名字空间的内容自动合并，同名的名字空间就是同一个作用域，如果有命名冲突编译就会报错。

```
#include <iostream>
using namespace std;

namespace ns {
    int num1 = 1234;
}

namespace ns {
    int num2 = 5678;
    int num1 = 6666;
}

int main(int argc,const char* argv[])
{
    using namespace ns;
    cout << num1 << " " << num2 << endl;
```

```
    return 0;
}
```

2、名字空间中可以再定义名字空间

```
#include <iostream>
using namespace std;

namespace ns1
{
    namespace ns2
    {
        namespace ns3
        {
            int num = 123456789;
        }
    }
}

// 嵌套后的名字空间使用起来比较长，可以给内部名字空间取别名
namespace ns123=ns1::ns2::ns3;

int main(int argc,const char* argv[])
{
    cout << ns1::ns2::ns3::num << endl;
    return 0;
}
```

5、匿名名字空间

C++把默认的全局作用域当作一个没有名字的名字空间，也叫匿名名字空间，当全局的标识符被屏蔽，可以使用匿名空间的域限定符来指定全局作用域的标识符。

```
#include <iostream>
using namespace std;

int num = 1234;
int main(int argc,const char* argv[])
{
    int num = 5678;
    cout << num << endl;
    cout << ::num << endl;
    return 0;
}
```

6、注意

工作时尽量不要使用 using namespace 空间名; 这种用法，这就像是把垃圾分类之后，又合并了，如果使用到全局标识符，一定要 空间名::标识符。

九、面向过程与面向对象

1、什么是面向过程编程

在编程时重点考虑如何解决问题，以及解决问题的具体步骤。

2、什么是面向对象编程

在编程时重点考虑的是"谁"能解决问题（类、结构），以及"它"解决问题时所需要属性（成员变量）和功能（成员函数）。

抽象：

把解决“问题者”当作思考或观察对象，把解决问题所需的具体的属性和功能罗列出来，这个步骤叫作抽象。

封装：

把抽象的结果（问题所需的具体的属性和功能），设置相应访问权限（public/private/protected），按照C++的语法设计成类、结构，该过程叫作封装类，简称封装。

使用封装好的类、结构实例化出对象（定义结构变量、类变量），通过对象调用相关功能（成员函数）配合相关属性（成员变量）达到解决问题的目的。

继承：

1、在解决一个问题之前，先考虑现有的类是否能解决部分问题，如果有则继承该类，在此基础上进行扩展，以缩短解决问题的目的，代码复用。

2、在解决一个复杂庞大的问题时，把问题拆分成若干个小问题，每个小问题实现一个类去解决，最后把这若干个类通过继承进行汇总，达到解决问题的目的，这个方式可以降低问题的规模在难度，也方便团队分工、协作。

多态：

语句的多种形态，同样的语句根据环境或参数的不同，有不同的功能，这就叫多态。

多态可以分为：

编译时多态，编译器在编译代码时根据语句的参数、环境不同，能翻译出具有不同功能的二进制指令，例如：调用重载的函数，cin、cout的自动类型识别，模板技术（后续讲解）。

运行时多态，语句只在程序运行时，才能解决执行那种功能，一般指的是类多态，后续再详细讲解。

```
#include <iostream>
using namespace std;

void func1(void)
{
    cout << "我是func1函数" << endl;
}

void func2(void)
{
    cout << "我是func2函数" << endl;
}

int main()
{
```

```
    void (*fp)(void);

    int cmd;
    cin >> cmd;
    if(cmd % 2)
        fp = func1;
    else
        fp = func2;

    fp();
}
```

十、类和对象

1、什么是类

把抽象结果（利用面向对象的思维模式，思考、观察出的结果），使用用C++的语法封装出一种类似结构的自定义数据类型（复合数据类型）。

2、如何设计类

```
struct 结构名
{
    成员函数;   // 结构的成员默认访问权限是public
private:
    成员变量;
};

class 类名
{
    成员变量;   // 结构的成员默认访问权限是private
public:
    成员函数;
};

注意：在C++中类的结构除了成员的默认访问权限不同，没有任何区别(大师兄说的)。
```

3、访问限制符

使用以下关键字修改结构、类的成员，管理它们的访问权限：

```
public: 公开的，被修饰的成员，可以在任何位置使用。
private: 私有的，被它修饰的成员，只能在类的成员函数内使用。
protected: 保护的，被它修饰的成员，只能在类、子类的成员函数内使用。
```

```
#include <iostream>
using namespace std;

class Student
{
    int id;
    char name[20];
    short age;
    float score;
```

```
public:
    void in(void)
    {
        cout << "请输入学生的学号、姓名、年龄、成绩: ";
        cin >> id >> name >> age >> score;
    }

    void out(void)
    {
        // 学号:10010 姓名:hehe 年龄:18 成绩:100
        cout << "学号:" << id << " 姓名:" << name << " 年龄:" << age << " 成绩:" << score << endl;
    }
};

int main(int argc,const char* argv[])
{
    Student stu;
    stu.in();
    stu.out();
    return 0;
}
```

4、什么是类对象

使用设计好的类这种数据类型，定义出的类变量在面向对象编程语言中被称为对象，创建类对象的行为也被称为实例化对象。

```
#include <iostream>
using namespace std;

class Student
{
    int id;
    char name[20];
    short age;
    float score;
public:
    void in(void)
    {
        cout << "请输入学生的学号、姓名、年龄、成绩: ";
        cin >> id >> name >> age >> score;
    }
    void out(void)
    {
        // 学号:10010 姓名:hehe 年龄:18 成绩:100
        cout << "学号:" << id << " 姓名:" << name << " 年龄:" << age << " 成绩:" << score << endl;
    }
};

Student stu; // 在bss内存段上实例化对象
int main(int argc,const char* argv[])
{
    Student stu; // 在stack内存段实例化对象
    Student* stup = new Student; // 在heap内存段实例化对象
}
```

5、类的声明与实现分开

如果类的内容不多，一般会选择只定义一个头文件，以类名作为文件名。

```
#ifndef STUDENT_H
#define STUDENT_H

class 类名
{
    // 定义成员变量
public:
    // 定义成员函数
};

#endif//STUDENT_H
```

如果类的内容比较多，会选择把类的声明与定义分开，头文件负责类的声明，源文件负责类的实现。

```
#ifndef STUDENT_H
#define STUDENT_H

class 类名
{
    // 定义成员变量
public:
    // 声明成员函数
    void 函数名(参数列表);
};

#endif//STUDENT_H
```

```
#include "student.h"

void 类名::函数名(参数列表)
{

}
```

十一、构造函数与析构函数

构造函数

1、什么构造函数

类、结构、联合中的特殊成员函数，与类名、结构名、联合名同的成员函数，没有返回值。

```
class 类名
{
public:
    类名(参数列表)
    {

    }
};
```

2、何时调用构造函数

当创建类对象时(实例化对象)会自动调用构造函数。

```
int main(int argc,const char* argv[])
{
    // 调用无参构造
    Student stu;
    Student* stup = new Student;

    // 调用有参构造
    Student stu1(10010,"hehe",'w',18,98);
    Student* stup1 = new Student(10011,"xixi",'m',20,95);

    // 调用有参构造，c11语法标准才支持，-std=c++11
    Student stu2 = {10010,"hehe",'w',18,98};
    return 0;
}
```

3、在构造函数中应该做什么

构造函数负责创建使用对象的环境，一般构造函数中负责：

1、给成员变量初始化

2、给指针成员分配内存

3、从文件、网络、数据库加载数据

4、完成一些准备工作

4、实现构造函数要注意的问题

在创建类对象时，一定会调用构造函数，如果类中没有显式实现构造函数，编译器会自动生成一个无参的什么都不做的构造函数。

如果显式实现了有参构造函数，那么编译器就不会再生成无参构造。

注意：在使用new[] 创建n个对象时，无法保证给所有的类对象都单独提供参数，去调用有参构造，那么必然需要调用无参构造，此时如果没有显式实现无参构造，编译就会出错。

方法1：

```
class Student
{
public:
    Student() {} // 极简无参构造
};
```

方法2：

给有参构造函数的所有成员都设置默认形参，当需要调用无参构造时，会自动使用默认参数调用有参构造。

```
// 无参构造未必无参
class Student
{
    ...
public:
    Student(int _id=0,const string& _name="",char _sex='w',short _age=0,float _score=0)
    {
        id = _id;
        name = _name;
        sex = _sex;
        age = _age;
        score = _score;
        cout << "我是有参构造" << endl;
    }
};
```

5、explicit关键字的作用

如果类中有单参的构造函数，那么它的参数类型数据就能隐藏提升为类对象，如果不使用该功能，可以在单参构造函数的前面加 explicit 关键字，参数类型数据就不能再隐藏提升为类对象。

```
#include <iostream>
using namespace std;

class Test
{
    int num;
public:
    [explicit] Test(int _num)
    {
        num = _num;
    }
};

void func(Test t)
{
    cout << "我被调用了" << endl;
}

int main(int argc,const char* argv[])
{
    func(1234);
}
```

析构函数：

1、什么析构函数

类、结构、联合中的特殊成员函数，在类名、结构名、联合的前面加~，就是析构函数，没有返回也没有参数。

```
struct Student
{
    int id;
    string name;
    char sex;
    short age;
    float score;
public:
    Student(void)
    {
        cout << "我是无参构造函数" << endl;
    }
    Student(int _id,const string& _name,char _sex,short _age,float _score)
    {
        id = _id;
        name = _name;
        sex = _sex;
        age = _age;
        score = _score;
        cout << "我是有参构造" << endl;
    }
    ~Student(void)
    {
        cout << "我是析构函数" << endl;
    }
};
```

2、在析构函数中应该做什么

析构函数负责对象销毁时的收尾工作，一般构造函数中负责：

1、释放指针成员指向的堆内存

2、往文件中、数据库保存数据

3、何时调用析构函数

1、当类对象离开它所在的作用域时，系统会自动销毁类对象，并自动调用析构函数。

```
#include <iostream>
using namespace std;

struct Student
{
    int id;
    string name;
    char sex;
    short age;
    float score;
public:
    Student(void)
```

```
    {
        cout << "我是无参构造函数" << endl;
    }
    Student(int _id,const string& _name,char _sex,short _age,float _score)
    {
        id = _id;
        name = _name;
        sex = _sex;
        age = _age;
        score = _score;
        cout << "我是有参构造" << endl;
    }
    ~Student(void)
    {
        cout << "我是析构函数" << endl;
    }
};

int main(int argc,const char* argv[])
{
    do{
        // 当创建类对象时，自动调用构造函数，当对象离开它的作用域时，会自动调用析构函数
        Student stu;
    }while(0);
    cout << "-----------" << endl;
}
```

2、当使用delete或delete[] 释放堆内存上的类对象时，会自动调用析构函数。

```
#include <iostream>
using namespace std;

struct Student
{
    int id;
    string name;
    char sex;
    short age;
    float score;
public:
    Student(void)
    {
        cout << "我是无参构造函数" << endl;
    }
    Student(int _id,const string& _name,char _sex,short _age,float _score)
    {
        id = _id;
        name = _name;
        sex = _sex;
        age = _age;
        score = _score;
        cout << "我是有参构造" << endl;
    }
    ~Student(void)
    {
        cout << "我是析构函数" << endl;
    }
```

```
};

int main(int argc,const char* argv[])
{

    // 使用new 或 new[] 在堆内存上创建类对象时会自动调用构造函数
    Student* stup = new Student;
    Student* stus = new Student[10];

    // 类对象的个数，也是执行构造、析构函数的次数
    cout << *((int*)stus-1) << endl;

    // 使用delete 或 delete[] 释放堆内存上的类对象时会自动调用析构函数
    delete stup;
    delete[] stus;

    // 注意：使用new创建的就使用delete释放，使用new[]创建的就使用delete[]释放，不能混用
    return 0;
}
```

4、析构函数要注意的问题

如果没有显式实现析构函数，那么编译器会自动生成一个什么都不做的析构函数。

如果构造函数中使用new、malloc分配的堆内存，就一定要显式实现析构函数，并在析构函数中释放堆内存，否则就会造成内存泄漏。

十二、初始化列表与const成员

1、const成员

使用const修饰的类、结构、联合的成员变量，在类对象创建完成前一定要初始化。

不能在构造函数中初始化const成员，因为执行构造函数时，类对象已经创建完成，只有类对象创建完成才能调用成员函数，构造函数虽然特殊但也是成员函数。

在定义const成员时进行初始化，该语法只有在C11语法标准下才支持。

2、初始化列表

在构造函数小括号后面，主要用于给const成员初始化的一种特殊语法，也可以给普通成员初始化（可以解决参数名与成员变量名同名的问题）。

注意：如果有成员是类、结构、联合类型的，还可以在初始化列表中显式调用成员的构造函数，还可以调用父类的构造函数。

```
class 类名
{
    成员变量;
public:
    构造函数(参数列表):成员变量(初始值),成员变量(初始值),...
    {

    }
};
```

```cpp
#include <iostream>
using namespace std;

class Test
{
    // 可以使用，但只有C11标准才支持
    // const int num = 1234;
    const int num;
public:
    Test(int num=0):num(num)
    {
        cout << "我是带初始化列表的构造函数" << endl;
    }
    void show(void)
    {
        cout << "const num = " << num << endl;
    }
};
int main(int argc,const char* argv[])
{
    Test t(123456789);
    t.show();
    return 0;
}
```

十三、this指针与常函数

1、成员函数是如何区别调用它的对象?

```cpp
#include <iostream>
using namespace std;

class Test
{
    int num;
public:
    Test(int num):num(num) {}
    void show(void)
    {
        cout << num << " " << &num << endl;
    }
};

int main(int argc,const char* argv[])
{
    Test t1(1234), t2(5678);
    t1.show();
    t2.show();
}
```

C语言中我们如何解决该问题：

由于C语言中的类无法有成员函数，只能定义普通函数，取函数名时，让它与结构有关联，然后在函数的第一个参数把结构变量的地址传递过来，从而区别每个调用它的结构变量。

在C++语言中该问题由编译器帮忙解决：

1、C++语言中的结构、联合、类可以定义成员函数，成员函数都会有一个隐藏的参数。

2、该参数就是个地址，也叫this指针，可以显式使用。

3、使用结构、联合、类对象可以直接调用成员函数时，编译器会自动计算对象的地址隐式的传递给成员函数。

4、所以在成员函数中可以区分是哪个对象调用了它，并且在成员函数内访问成员变量时，编译器帮我们隐式在每个成员变量前增加了->，所以可以区别出每个对象的成员变量。

5、由于成员函数参数列表中都隐藏着this指针，所以普通的成员函数无法作为回调用函数的参数。

```
#include <iostream>
#include <pthread.h>
#include <signal.h>
using namespace std;

class Test
{
    int num;
public:
    Test(int num):num(num){ }

    void* run(void* arg)
    {

    }

    void sigint(int signum)
    {

    }

    void show(void)
    {
        cout << num << " " << &num << endl;
    }
};

int main(int argc,const char* argv[])
{
    signal(SIGINT,Test::sigint);

    pthread_t tid;
    pthread_create(&tid,NULL,Test::run,NULL);

    return 0;
}
```

6、显式使用this指针可以解决函数参数与成员变量同名的问题。

```
#include <iostream>
#include <pthread.h>
#include <signal.h>
using namespace std;

class Test
{
    int num;
public:
    Test(int num):num(num){ }
    void setNum(int num)
    {
        this->num = num;
        cout << this->num << " - " << &this->num << endl;
    }

    void show(void)
    {
        cout << num << " " << &num << endl;
    }
};

int main(int argc,const char* argv[])
{
    Test t1(1234), t2(5678);
    t1.show();
    t2.show();
    t1.setNum(6666);
    t1.show();
}
```

2、常对象与常函数

什么是常函数

在成员函数的参数列表的末尾（小括号后面），用const修饰，这种成员就叫常函数。

```
class 类名
{
public:
    // 常函数
    返回值 函数名(参数列表) const
    {

    }
};
```

常函数的特点：

常函数隐藏的this指针具有cosnt属性。

什么是常对象

在定义结构、联合、类对象时，使用const修饰，这种对象就叫常对象。

```
const 类名 对象名;
const 类名* 指针变量 = new 类名;
```

常对象的特点：

使用常对象调用成员函数时，编译器计算出的对象地址具有const属性。

常对象有常函数的局限性：

1、常对象不能调用普通成员函数，只能调用常函数（构造函数和析构函数除外），但普通对象既可以普通成员函数，也可以调用常函数。

2、在常函数中不能显式修改成员变量，并且也不能调用普通的成员函数，只能调用常函数。

3、如果类的对象一定会被const修饰，那么它的成员函数都要定义为常函数。

4、如果类的对象可能被const修饰，也可能不修饰，那么它的成员函数要写两份，一份常函数，另一份普通成员函数。

3、mutable关键字的作用

如果常函数的const属性与函数的功能发生冲突，一定要修改成员变量，那么使用mutable关键字修饰一下需要在常函数中修改的成员变量。

```
#include <iostream>
#include <pthread.h>
#include <signal.h>
using namespace std;

class Test
{
    mutable int num;
public:
    Test(int num):num(num){ }

    void setNum(int num) const
    {
        this->num = num;
        cout << this->num << " - " << &this->num << endl;
    }

    void show(void) const
    {
        cout << num << " " << &num << endl;
    }
};

int main(int argc,const char* argv[])
{
    const Test t(1234);
    t.show();
    t.setNum(23456);
    t.show();

    return 0;
}
```

4、空类的对象为什么占用1字节内存?

1、因为C++中的空的结构、联合、类里面有隐藏的成员函数。

2、成员函数的参数列表中有隐藏this指针，调用这些成员函数时就需要计算出对象的地址传递给成员函数。

3、空的类对象至少要在内存中占据一个字节，才可以计算出this指针，传递成员函数，完成函数调用。

十四、拷贝构造函数和赋值函数

1、什么是拷贝构造

是一种特殊构造函数，如果没有显式的实现，编译器就会自动生成。

```
class 类名
{
public:
    // 拷贝构造
    类名(类名& that)
    {

    }
};
```

2、什么会调用拷贝构造

当使用一个类对象给另一个新的类对象初始化时，就会自动调用拷贝构造。

```
#include <iostream>
using namespace std;

class Test
{
public:
    Test(void)
    {
        cout << "调用了普通的构造函数" << endl;
    }
    Test(const Test& that)
    {
        cout << "调用了拷贝构造" << endl;
    }
};

void func(Test t)
{

}

Test func(void)
{
    Test t;
    ...
```

```
    return;
}

int main(int argc,const char* argv[])
{
    Test t1;            // 调用的是普通构造
    Test t2 = t1;       // 调用的是拷贝构造
    Test t3 = func()    // 编译器会进行优化，直接把t和使用权转交给t3，不会调用拷贝构造
    func(t1);           // 调用的是拷贝构造

    return 0;
}
```

3、拷贝构造的任务是什么

拷贝构造参数对象的所有成员变量挨个赋值给新对象的成员变量，一般情况下编译器自动生成的拷贝构造就能完全满足我们使用需求。

4、什么时候需要显式实现拷贝构造

当成员变量中有指针成员且指向了堆内存，就需要显式实现拷贝构造。

编译器自动生成的拷贝构造，只会对成员变量挨个赋值，如果成员员中有指针变量且指向堆内存，结果就两个对象的指针变量同时指向一份堆内存，当它们执行析构函数时，会把这块堆内存释放两次，产生 **double free or corruption** 的错误。

正确的做法应该是先给新对象的指针变量重新申请一份堆内存，然后把旧对象的指针变量所指向的内存拷贝到新对象的指针变量所指向的内存。

```
#include <iostream>
using namespace std;

class Test
{
    int* ptr;
public:
    Test(int num)
    {
        ptr = new int;
        cout << "new:" << ptr << endl;
        *ptr = num;
    }

    ~Test(void)
    {
        cout << "delete:" << ptr << endl;
        delete ptr;
    }

    /* 编译器生成的拷贝构造，会造成 double free
    Test(const Test& that)
    {
        ptr = that.ptr;
    }
    */
    Test(const Test& that)
```

```
    {
        // 给新对象的指针变量重新申请堆内存
        ptr = new int;
        // 把旧对象的指针变量所指向的内存拷贝给新对象的指针变量所指向的内存，如果不方便解引用时可以使用memcpy函数
        *ptr = *that.ptr;
    }

    void show(void)
    {
        cout << "val:" << *ptr << " addr:" << ptr << endl;
    }
};

int main(int argc,const char* argv[])
{
    Test t1(12345);
    Test t2 = t1;
    t1.show();
    t2.show();

    return 0;
}
```

5、什么是赋值函数

是一种特殊的成员函数，如果没有显式实现，编译器会自动生成。

```
class 类名
{
public:
    // 赋值函数
    const 类名& operator=(类名& that)
    {

    }
};
```

6、什么时候会调用赋值函数

当一个旧对象给另一个旧对象赋值时会自动调用赋值函数。

当一个旧对象给另一个新对象初始化时会自动调用拷贝构造函数。

```
#include <iostream>
using namespace std;

class Test
{
public:
    Test(const Test& that)
    {
        cout << "调用了拷贝构造" << endl;
    }

    void operator=(const Test& that)
    {
```

```
        cout << "调用了赋值函数" << endl;
    }
};

int main(int argc,const char* argv[])
{
    Test t1;        // 调用了普通的构造函数
    Test t2 = t1;   // 调用了拷贝构造
    t1 = t2;        // 调用的是赋值函数
    return 0;
}
```

7、赋值函数的任务是什么

赋值函数与拷贝构造的任务几乎相同，都是挨个给成员变量赋值，但如果需要显式实现时，它的业务逻辑不同。

8、什么时候需要显式实现赋值函数

当需要显式实现拷贝构造时，就需要显式实现赋值函数，它们两个面临问题是一样的。

赋值函数不应该对成员指针变量赋值，而应该对象成员指针变量所指向的内存进行拷贝。

```
#include <iostream>
using namespace std;

class Test
{
    int* ptr;
public:
    Test(int num)
    {
        ptr = new int;
        cout << "new " << ptr << endl;
        *ptr = num;
    }
    ~Test(void)
    {
        cout << "delete " << ptr << endl;
        // delete ptr;
    }

    Test(const Test& that)
    {
        ptr = new int;
        // 如果不方便解引用，可以调用memcpy函数进行拷贝
        *ptr = *that.ptr;
        cout << "new " << ptr << "调用了拷贝构造" << endl;
    }

    const Test& operator=(const Test& that)
    {
        // 当ptr和that.ptr指向的内存块大小一样，可以直接进行内存拷贝
        *ptr = *that.ptr;
        cout << "调用了赋值函数" << endl;
        return *this;
        /*
```

```
            当对象的ptr指向的内存与与that.ptr指向的内存块不一样大
            先释放旧的ptr
            再分配新的，要与that.ptr的内存块一样大
            然后再拷贝
        */
    }
};

int main(int argc,const char* argv[])
{
    Test t1(1234);       // 调用了普通的构造函数
    Test t2 = t1;    // 调用了拷贝构造
    t1 = t2;         // 调用的是赋值函数
    return 0;
}
```

9、浅拷贝与深拷贝

拷贝就是一个对象给另一个对象赋值，编译器自动生成的拷贝构造和赋值函数执行的业务逻辑就是浅拷贝(成员指针给成员指针赋值)，深拷贝就是把成员指针所指向的内存拷贝给另一个成员指针所指向的内存。

浅拷贝就是指针给指针赋值，深拷贝就内存给内存赋值。

注意：如果成员变量中没有成员指针，则浅拷贝就可以满足需求，如果如果成员变量中有成员指针且指向堆内存，则必须显式实现深拷贝，否则就会出现 **double free or corruption** 的错误。

10、自定义MyString类

实现构造、析构、拷贝构造、赋值等函数。

```
#include <iostream>
#include <string.h>
using namespace std;

class MyString
{
    char* str;
public:
    MyString(const char* str="")
    {
        int len = strlen(str);
        this->str = new char[len+1];
        strcpy(this->str,str);
    }

    MyString(const MyString& that)
    {
        int len = strlen(that.str);
        str = new char[len+1];
        strcpy(str,that.str);
    }

    const MyString& operator=(const MyString& that)
    {
        if(this == &that)
```

```
            return *this;

        delete str;
        int len = strlen(that.str);
        str = new char[len];
        strcpy(str,that.str);
        return *this;

        /*
        MyString tmp = that;
        swap(tmp.str,this->str);
        return *this;
        */
    }

    ~MyString(void)
    {
        delete[] str;
    }

    void show(void)
    {
        cout << str << endl;
    }
};

int main(int argc,const char* argv[])
{
    // 构造函数
    MyString str = "hehe";
    str.show();

    // 拷贝构造
    MyString str1 = str;
    str1.show();

    // 构造函数
    MyString str2 = "xixix" , str3 = "大师兄";

    // 赋值函数
    str2 = str3;
    str2.show();

    return 0;
}
```

十五、静态成员

1、普通成员

普通成员变量的特点：

每创建一个对象，就分给该对象分配一块内存，里面存储成员变量，每多一个对象就多一份成员变量。

普通成员函数的特点：

成员函数的参数列表中隐藏一个this指针，当通过对象调用成员函数时，编译器会自动计算出对象的地址隐式的传递给this。

只能通过类对象才能调用成员函数。

2、静态成员

什么是静态成员变量：

被static修饰过的成员变量叫静态成员变量。

```
class Test
{
    // 静态成员
    static int num;
public:
    void show(void)
    {
        cout << num << endl;
    }
};
```

静态成员变量的特点和局限性：

1、静态成员变量只能在类内声明，定义和初始化必须放在类外。

2、静态成员变量使用的是data或bss内存段，所以类中的静态成员变量只有一份，所有类对象共用这一份静态成员变量。

3、如果类中有静态成员变量，计算类对象字节数时，静态成员变量不包含在内。

什么是静态成员函数：

被static修饰的成员函数叫静态成员函数。

```
class Test
{
    static int num;
public:
    void show(void)
    {
        cout << num << " " << &num << endl;
    }
    static void func(void)
    {

    }
};
```

静态成员函数的特点和局限性：

1、静态成员函数的参数列表中没有隐藏的this指针。

2、静态成员函数中不能直接访问成员，也不能调用其它成员函数，但可以访问静态成员也可以调用其它静态成员函数。

3、静态成员函数可以使用 类名::函数名(实参) 访问，不需要通过类对象就可以调用，但也可以通过类对象调用，但能访问直接对象的成员变量。

4、如果把类对象以参数形式，传递到静态成员函数中，那么它就可以访问私有的成员变量。

静态成员的作用

1、静态成员变量就相当于把普通全局变量的作用域限制到类内，如果它的访问权限是public，就可以当全局变量使用。

2、可以把类对象的共用成员设置为静态员，这样可以达到节约内存的目的，也可以作为类的管理信息。

3、静态成员函数相当把普通函数的作用域限制到类内，当作给所有对象提供了一个统一的管理接口，可以不破坏类的封装性提前访问静态成员对类对象进行管理和设置(管理信息就是私有的静态变量)。

十六、单例模式

1、什么是单例模式

只能创建出一个对象的结构、联合、类叫单例模式。

2、什么时候需要使用单例模式

1、Windows系统的任务管理器

2、网络或服务器程序的访问计数器。

3、线程池、数据池，一个程序中只能创建一个线程池或数据池。

3、单例模式的实现原理

1、把构造函数、拷贝构造设置成私有的，不允许创建类对象。

2、在类中声明定义一个静态的类对象或类对象指针。

3、提供一个静态成员接口，用于获取静态的类对象。

4、饿汉单例模式

只要程序一运行，就把单例类的类对象创建出来，不管后续是否使用到。

缺点：如果后续程序使用不到单例对象，就造成了资源、时间上的浪费。

优点：线程安全，再多的线程也不可能创建出多个类对象。

```
#include <iostream>
using namespace std;

class HungrySingle
{
    int num;
    static HungrySingle single;

    HungrySingle(int num = 1234)
    {
        this->num = num;
    }

    HungrySingle(const HungrySingle& that)
```

```
    {

    }
public:
    static HungrySingle& getSingleObject(void)
    {
        return single;
    }
    void show(void)
    {
        cout << num << " " << &num << endl;
    }
};

HungrySingle HungrySingle::single;

int main(int argc,const char* argv[])
{
    // HungrySingle s;
    // HungrySingle* s = new HungrySingle;
    HungrySingle& s1 = HungrySingle::getSingleObject();
    s1.show();
    HungrySingle& s2 = HungrySingle::getSingleObject();
    s2.show();
}
```

5、懒汉单例模式

直到调用获取单例类对象的接口才会把对象创建出来。

缺点：如果在多线程情况下，多个线程同时调用对象的获取接口，可能会创建出多个类对象。

优点：如果用不到单例对象就不会真正创建，节约资源和时间。

```
#include <iostream>
using namespace std;

class LazySingle
{
    int num;

    static LazySingle* single;

    LazySingle(int num = 0)
    {
        this->num = num;
    }
    LazySingle(const LazySingle& that)
    {

    }
public:
    static LazySingle& getSingleObject(void)
    {
        if(NULL == single)
        {
```

```
            single = new LazySingle();
        }
        return *single;
    }
    void show(void)
    {
        cout << num << " " << &num << endl;
    }
};

LazySingle* LazySingle::single;


int main(int argc,const char* argv[])
{
    // LazySingle s;
    // LazySingle* s = new LazySingle;

    LazySingle& s1 = LazySingle::getSingleObject();
    s1.show();

    LazySingle& s2 = LazySingle::getSingleObject();
    s2.show();
    return 0;
}
```

6、线程安全的懒汉单例

```
#include <iostream>
#include <pthread.h>
using namespace std;

class LazySingle
{
    int num;
    static pthread_mutex_t lock;
    static LazySingle* single;

    LazySingle(int num = 0)
    {
        this->num = num;
    }
    LazySingle(const LazySingle& that)
    {

    }
public:
    static LazySingle& getSingleObject(void)
    {
        pthread_mutex_lock(&lock);
        if(NULL == single)
        {
            single = new LazySingle();
        }
        pthread_mutex_unlock(&lock);
        return *single;
    }
```

```
    void show(void)
    {
        cout << num << " " << &num << endl;
    }
};

LazySingle* LazySingle::single;
pthread_mutex_t LazySingle::lock = PTHREAD_MUTEX_INITIALIZER;

int main(int argc,const char* argv[])
{
    // LazySingle s;
    // LazySingle* s = new LazySingle;

    LazySingle& s1 = LazySingle::getSingleObject();
    s1.show();

    LazySingle& s2 = LazySingle::getSingleObject();
    s2.show();
    return 0;
}
```

十七、运算符函数

1、什么是运算符函数

C++中把运算符当作函数处理，在表达式中遇到运算符时，编译器会先寻找相关的运算符函数。

2、全局运算符函数

例如a+b，会先寻找 **[const]T[&] operator+([const] T& a,[const] T& b) [const]** 格式的函数，这种函数又被称为全局运算符函数，运算对象都做为参数传递给该函数。

```
#include <iostream>
using namespace std;

enum INT {
    VAL = 1
};

int operator+(const int& a,INT& b)
{
    return b-a;
}

int main(int argc,const char* argv[])
{
    int a = 3;
    INT b = VAL;
    cout << a+b << endl;
    return 0;
}
```

3、成员运算符函数

如果运算对象是自定义类型，还可以把运算符函数实现该类型的内部，a+b的运算符函数格式为**[const] T[&] operator+([const]T& that) [const]**，这种函数又被称为成员运算符函数，一般由左边的运算对象发起调用，右边的运算对象作为参数。

```
#include <iostream>
using namespace std;

class INT
{
    int val;
public:
    INT(int val):val(val) {}
    int operator+(const int& that) const
    {
        cout << "-----------" << endl;
        return val + that*10;
    }
};

int main(int argc,const char* argv[])
{
    INT a = 3;
    int b = 7;

    cout << a+b << endl;
    return 0;
}
```

十八、运算符重载

在设计结构、联合、类时，为了让它们的对象的操作更简便、易懂，我们会为它们实现部分运算符函数，这种行为被称为运行符重载，我们以自定义string为例进行讲解运算符重载。

1、双目运算符重载

重载为成员运算符函数：

```
#include <iostream>
#include <string.h>
using namespace std;

class MyString
{
    char* str;
public:
    MyString(const char* str="")
    {
        this->str = new char[strlen(str)+1];
        strcpy(this->str,str);
    }

    MyString(const MyString& that)
    {
        str = new char[strlen(that.str)+1];
```

```
        strcpy(str,that.str);
    }

    ~MyString(void)
    {
        delete[] str;
    }

    const MyString&  operator=(const MyString& that)
    {
        cout << "赋值函数" << endl;
        if(this == &that)
            return *this;

        delete[] str;
        str = new char[strlen(that.str)+1];
        strcpy(str,that.str);
        return *this;
    }

    const MyString operator+(const MyString& that) const
    {
        MyString tmp;
        delete tmp.str;

        tmp.str = new char[strlen(str)+strlen(that.str)+1];
        strcpy(tmp.str,str);
        strcat(tmp.str,that.str);
        return tmp;
    }

    MyString& operator+=(const MyString& that)
    {
        MyString tmp = *this+that;
        swap(tmp.str,str);
        return *this;
    }

    bool operator==(const MyString& that) const
    {
        return 0 == strcmp(that.str,str);
    }

    bool operator!=(const MyString& that) const
    {
        return 0 != strcmp(that.str,str);
    }

    bool operator>(const MyString& that) const
    {
        return 0 > strcmp(that.str,str);
    }

    bool operator<(const MyString& that) const
    {
        return 0 < strcmp(that.str,str);
    }

```

```
    bool operator>=(const MyString& that) const
    {
        return 1 != strcmp(that.str,str);
    }

    bool operator<=(const MyString& that) const
    {
        return -1 != strcmp(that.str,str);
    }

    const char operator[](size_t i) const
    {
        return str[i];
    }

    char& operator[](size_t i)
    {
        return str[i];
    }

    void show(void) const
    {
        cout << str << endl;
    }
};

int main(int argc,const char* argv[])
{
    /*
    测试构造函数
    MyString str1 = "1234";
    str1.show();
    MyString str2("12345");
    str2.show();
    */

    // 拷贝构造、赋值函数
    MyString str1("1234"),str2("4567");
    MyString str3 = str1;
    str3.show();
    str3 = str2;
    str3.show();

    cout << (str1 == str2) << endl;
    cout << (str3 == str2) << endl;
    cout << (str1 != str2) << endl;
    cout << (str3 != str2) << endl;
    cout << (str1 < str2) << endl;
    cout << (str1 > str2) << endl;

    // 调用的赋值函数，因为有单参构造，可以进行类型提升,explicit
    str2 = "12345";
    cout << (str1 >= str2) << endl;
    cout << (str1 <= str2) << endl;

    str2[4] = 'B';
    cout << str2[4] << endl;

```

```
    const MyString cstr = "abcdefg";
    cout << cstr[0] << endl;

    return 0;
}
```

1、左边的运算对象如果可能是常量，则需要把运算符函数设置为常函数。

2、如果右边的运算对象可能是常量，则需要给形参设置const属性。

3、一般在运算符函数中不修改运算对象的数据，而是由左右的运算对象相结构产生临时对象。

重载为全局运算符函数：

当无法修改对象的源码时，也就无法在结构、联合、类的内部增加成员函数，又想给类对象增加运行符函数，这种情况可以定义全局的运算符，把运算对象都作为参数传递给运算符函数。

```
#include <iostream>
#include <string.h>
using namespace std;

bool operator/(const string& str1,const string& str2)
{
    return NULL != strstr(str1.c_str(),str2.c_str());
}

int main(int argc,const char* argv[])
{
    string str1 = "123456789" ,str2 = "456";
    cout << str1/str2 << endl;
    return 0;
}
```

友元函数：

当一个非成员函数，需要访问类中的私有成员时，可以把函数设置为该类的friend函数（也叫友元函数），这样在函数内就可以访问类的所有成员变量和成员函数。

一般情况下，在类的内部进行声明该函数，并在声明的前面添加friend关键字，在类外实现友元函数。

也可以直接在类的内部实现有友函数，即使在类的内部实现也依然不是成员函数。

当重载全局运算符函数基本上都需要使用友元函数，我个人建议把友元函数实现在类的内部。

```
#include <iostream>
using namespace std;

class Test
{
    int num;
    void func(void)
    {
        cout << "我是私有函数" << endl;
    }
public:
    Test(int num):num(num){}
```

```
    friend void test_friend1(Test& t);
    friend void test_friend2(Test& t)
    {
        t.func();
        cout << t.num << endl;
    }
};

void test_friend1(Test& t)
{
    t.func();
    cout << t.num << endl;
}

int main(int argc,const char* argv[])
{
    Test t(12345);
    test_friend1(t);
    test_friend2(t);
}
```

2、输入、输出运算符重载

1、cin、cout是标准库中具有输入、输出功能的类对象，它的类名叫istream、ostream，当我们想给某种类对象增加输入、输出的功能时，就需要实现 >>、<<运算符函数对于该类对象的支持，也就是重载输入、输出运算符。

2、由于cin、cout都在>>、<<运算的左边，如果想实现成员运算符函数，就需要在istream、ostream类中实现，而我们无法增加或修改istream、ostream类的代码，所以只能实现全局的 >>、<<运算符函数。

3、由于输入、输出过程中需要使用cin、cout记录成功、失败等状态，所以输入、输出函数的istream、ostream要使用引用，且不能用const修饰。

```
class MyString
{
    ...
    friend ostream& operator<<(ostream& os,const MyString& str)
    {
        return os << str.str;
    }
    friend istream& operator>>(istream& is,MyString& str)
    {
        size_t size = 0;
        do{
            str.str[size++] = is.get();
            str.str = (char*)realloc(str.str,size+1);
        }while(' ' != str.str[size-1] && '\n' != str.str[size-1]);

        str.str[size-1] = '\0';
        return is;
    }
};


```

3、单目运算符的重载

单目运算符的重载与又目运算符重载的规则几乎，只是运算对象数量不同而已，

重载为成员运算符函数：

唯一的运算对象就是函数的调用者，参数列表为空，**[cosnt] T[&] operator<单目运算符>(void) [const]。**

```
#include <iostream>
using namespace std;

class Test
{
    int num;
public:
    Test(int num = 0):num(num) {}
    friend istream& operator>>(istream& is,Test& t)
    {
        return is >> t.num;
    }
    friend ostream& operator<<(ostream& os,const Test& t)
    {
        return os << t.num;
    }

    const Test operator~(void)const
    {
        Test temp ;
        temp.num = ~num;
        return temp;
    }
    const Test operator-(void)const
    {
        Test temp ;
        temp.num = -num;
        return temp;
    }
    bool operator!(void)const
    {
        return  !num;
    }
};

int main(int argc,const char* argv[])
{
    Test t(0x7fffffff);
    cout << t << endl;
    cout << !t << endl;
}
```

重载为全局运算符函数：

与双目运算符一样，只有无法修改运算对象的代码时，才会重载全局运算符函数，参数列只有一个，**[cosnt] T[&] operator<单目运算符>([const] T&)[const]。**

一旦重载为全局运算符函数就需要在全局函数中访问对象的私有成员，但又无法在类对象中增加友元，理论上单目运算符支持重载为全局运算符函数，但实际情况往往不允许。

```cpp
#include <iostream>
using namespace std;

class Test
{
public:
    Test(int num = 0):num(num) {}
    friend istream& operator>>(istream& is,Test& t)
    {
        return is >> t.num;
    }
    friend ostream& operator<<(ostream& os,const Test& t)
    {
        return os << t.num;
    }
    int num;
};

const Test operator~(const Test& t)
{
    Test temp ;
    temp.num = ~t.num;
    return temp;
}
const Test operator-(const Test& t)
{
    Test temp ;
    temp.num = -t.num;
    return temp;
}
bool operator!(const Test& t)
{
    return  !t.num;
}

int main(int argc,const char* argv[])
{
    Test t(0x7fffffff);
    cout << t << endl;
    cout << !t << endl;
}
```

4、自变运算符重载

前自变运算符重载 ++i/--i：

前自变运算符与普通的单目运算符重载方法相同。

```cpp
class Test
{
    int num;
public:
    ...
    Test& operator++(void)
    {
        num++;
        return *this;
```

```
    }
    Test& operator--(void)
    {
        num--;
        return *this;
    }
};
```

后自变运算符重载 ++i/--i：

后自变运算符的运算对象也只有一个，为了区别前自变，我们需要在参数列增加一个什么都不做的int关键字，我们把这种用法叫**哑元**。

```
class Test
{
    int num;
public:
    ...
    Test operator++(int)
    {
        Test temp = *this;
        num++;
        return temp;
    }
    Test operator--(int)
    {
        Test temp = *this;
        num--;
        return temp;
    }
};
```

5、new/delete运算符的重载

在C++中是把new/delete关键字当作运算符，也就是运算符函数，所以如果有特殊需要可以对它们进行重载，重载格式参考单目运算符。

new运算符的重载：

new 运算符有两中用法，也就有两个格式重载函数。

格式1：new TYPE

该格式既可以重载为全局函数，也可以重载为成员函数。

```
class Test
{
    int num;
public:
    void* operator new(size_t size)
    {
        size = (size/4+1)*4;
        void* ptr = malloc(size);
        cout << "malloc " << ptr << " " << size << "byte" << endl;
        return ptr;
    }
};
```

```

void* operator new(size_t size)
{
    void* ptr = malloc((size/4+1)*4);
    cout << "new " << ptr << endl;
    return ptr;
}

int main(int argc,const char* argv[])
{
    Test* t = new Test;
}
```

格式2：new(ptr) TYPE

该格式只能重载为成员函数，因为C++标准库中已经为该格式的函数。

```
class Test
{
    int num;
public:
    void* operator new(size_t size,void* ptr)
    {
        size = (size/4+1)*4;
        ptr = realloc(ptr,size);
        cout << "realloc " << ptr << " " << size << "byte" << endl;
        return ptr;
    }
};

int main(int argc,const char* argv[])
{
    char* p = new char[40];
    Test* t = new(p) Test;
}
```

new/delete运算符有什么用：

1、记录下申请、释放的内存块的地址，方便程序员检测是否有内存泄漏。

2、防止用户大量分配、释放小块内存，产生内存碎片。

3、默认的new只能给类对象分配内存，重载后可以给成员指针一起分配内存，该方法适合重载为类的成员函数。

```
class Test
{
    int num;
    char* str;
public:
    ...
    void* operator new(size_t size)
    {
        // 减少内存碎片
        size = (size/4+1)*4;
        void* ptr = malloc(size);

```

```
        // 方便检查内存泄漏
        cout << "malloc " << ptr << " " << size << "byte" << endl;

        // 为成员指针分配内存
        ((Test*)ptr)->str = (char*)malloc(50);
        strcpy(((Test*)ptr)->str,"成员指针分配内存");

        return ptr;
    }

    void operator delete(void* ptr)
    {
        // 方便检测内存泄漏
        cout << "delete " << ptr << endl;
        // 释放成员指针指向的内存
        free(((Test*)ptr)->str);
        // 释放对象内存
        free(ptr);
    }
};
```

十九、智能指针

1、常规指针的缺点

当一个常规指针离开它的作用域时，只有该指针变量本身所占据的内存空间（4/8个字节）会被释放，而它所指向的动态内存并未得到释放，需要手动调用free/delete/delete[]去释放。

在某些特殊情况下，包含free/delete/delete[]的代码根据本就执行不到（压根没有写释放语句），就会形成内存泄漏。

2、智能指针的优点

智能指针是一个封装了常规指针的类类型对象，它里面有成员指针负责指向堆内存，当对象它离开作用域时，其析构函数负责释放成员指针所指向的动态内存。

以正确方式创建的智能指针，其析构函数肯定会执行。

3、智能指针与常规指针的相同点

为了使用智能指针也能像常规指针一样，通过"*"操作符解引用，通过"->"操作符访问其目标的成员，就需要对这两个操作符进行重载。

4、智能指针与常规指针的不同点

任何时候，针对一个对象，只允许有一个智能指针指向该内存，否则多个对象多次执行析构函数就会造成 **double free** 错误。

智能指针的拷贝构造和赋值函数需要做特殊处理。

5、简化版的智能指针类

```
#include <iostream>
using namespace std;

// int类型专用的智能指针
class AutopInt
{
```

```
    int* ptr;
public:
    AutopInt(int* ptr):ptr(ptr)
    {
        cout << "ptr = " << ptr << endl;
    }

    ~AutopInt(void)
    {
        cout << "delete " << ptr << endl;
        delete ptr;
    }

    int& operator*(void)
    {
        return *ptr;
    }
};

int main(int argc,const char* argv[])
{
    AutopInt p = new int(123456);
    cout << *p << endl;

    *p = 56789;
    cout << *p << endl;
    return 0;
}
```

```
#include <iostream>
using namespace std;

class Student
{
    int id;
    string name;
    char sex;
    float score;
public:
    Student(int id,const string& name,short sex,float score)
    {
        this->id = id;
        this->name = name;
        this->sex = sex;
        this->score = score;
        cout << "构造对象" << endl;
    }
    ~Student(void)
    {
        cout << "析构对象" << endl;
    }

    void setId(int id)
    {
        this->id = id;
    }

```

```
    void setName(const string& name)
    {
        this->name = name;
    }

    void setSex(char sex)
    {
        this->sex = sex;
    }

    void setScore(float score)
    {
        this->score = score;
    }

    int getId(void)
    {
        return id;
    }

    string getName(void)
    {
        return name;
    }

    char getSex(void)
    {
        return sex;
    }

    float getScore(void)
    {
        return score;
    }

    friend istream& operator>>(istream& is,Student& stu)
    {
        cout << "请输入学生的学号、姓名、性别、成绩：";
        return is >> stu.id >> stu.name >> stu.sex >> stu.score;
    }

    friend ostream& operator<<(ostream& os,Student& stu)
    {
        return os << stu.id << " " << stu.name << " " << stu.sex << " " << stu.score << endl;
    }
};

class AutopStudent
{
    Student* stup;
public:
    AutopStudent(Student* stup):stup(stup)
    {
        cout << "new " << stup << endl;
    }
    ~AutopStudent(void)
    {
```

```
        cout << "delete " << stup << endl;
        delete stup;
    }

    Student* operator->(void)
    {
        return stup;
    }

    Student& operator*(void)
    {
        return *stup;
    }
};

int main(int argc,const char* argv[])
{
    AutopStudent stup = new Student(10020,"大师兄",'m',20);
    //Student* stup = new Student(10030,"呵呵",'w',120);
    cout << stup->getName() << endl;
    cout << stup->getScore() << endl;
    cout << *stup << endl;
    AutopStudent stup2 = stup;
    //delete stup;
    return 0;
}
```

6、C++89标准中的智能指针auto_ptr

如何使用auto_ptr：

#include <memory>

auto_ptr <目标类型> 智能指针对象(new操作符返回的目标对象地址);

auto_ptr的基本用法：

```
int main(int argc,const char* argv[])
{
    auto_ptr<Student> stup(new Student(10010,"大师兄",'m',100));
    auto_ptr<Student> stup1 = stup;
    cout << *stup << endl;
    cout << stup->getScore() << endl;
    return 0;
}
```

auto_ptr的局限性：

1、不能跨作用域使用，因为它一旦出了作用域就会自动调用所指向对象的析构函数。

2、不能多个指针指向同一个对象。

3、不能指向对象数组，也就是不能指向连续的多个对象，也就是这种智能指针不能进行加减运算。

4、该指针基本没有被大量使用。

7、C++11中的新版智能指针

unique_ptr型智能指针：

unique_ptr 独占所指向的对象，一旦编译器发现智能指针之间赋值，就会报错，我们实现的AutopStudent智能指针要想达到同样的使用效果，只需要显式定义私有的拷贝构造。

```
class AutopStudent
{
    Student* stup;
    AutopStudent(const AutopStudent& that) { }
public:
    ...
};

int main(int argc,const char* argv[])
{
    AutopStudent stup(new Student(10020,"大师兄",'m',20));
    cout << stup->getName() << endl;
    cout << stup->getScore() << endl;
    cout << *stup << endl;

    AutopStudent stup2 = stup; // 编译时错
}

int main(int argc,const char* argv[])
{
    unique_ptr<Student> stup(new Student(10010,"大师兄",'m',100));
    unique_ptr<Student> stup1 = stup; // 编译时错
    cout << *stup << endl;
    cout << stup->getScore() << endl;
}
```

shared_ptr共享型智能指针：

shared_ptr 允许多个指针指向同一个对象，**shared_ptr** 增加了引用计数器，指针对象每拷贝一次，计算器就增加1，每执行一次构造计算器就减1，直到计算器为1时，执行析构函数，才把资源释放。

```
class AutopStudent
{
    Student* stup;
    size_t* cntp;
public:
    AutopStudent(Student* stup):stup(stup)
    {
        cout << "new " << stup << endl;
        cntp = new size_t(1);
    }
    AutopStudent(const AutopStudent& that)
    {
        stup = that.stup;
        cntp = that.cntp;
        *cntp += 1;
    }
    ~AutopStudent(void)
    {
        cout << "*cntp = " << *cntp << endl;
```

```
        if(*cntp > 1)
        {
            *cntp -= 1;
            return;
        }
        if(*cntp == 1);
        {
            cout << "delete " << stup << endl;
            delete stup;
            delete cntp;
        }
    }
    ...
};

void func(AutopStudent stup)
{
    cout << *stup << endl;
}

int main(int argc,const char* argv[])
{
    AutopStudent stup(new Student(10020,"大师兄",'m',20));
    cout << stup->getName() << endl;
    cout << stup->getScore() << endl;
    cout << *stup << endl;
    AutopStudent stup2 = stup;
    func(stup2);
}

```

weak_ptr弱引用型智能指针：

weak_ptr 只能作为**shared_ptr**的弱引用，也叫作**shared_ptr**指针的观察者，负责监控**shared_ptr**资源是否已经被释放，它不能单独使用，必须与**shared_ptr**指针配合使用，使用它的use_count()成员函数查看资源计算器。

```
shared_ptr<Student> stup(new Student(10111,"大师兄",'m',150));
weak_ptr<Student> wstup = stup;

void test_weak(void)
{
    cout << wstup.use_count() << endl;
    shared_ptr<Student> stup1 = stup;
    cout << wstup.use_count() << endl;
    shared_ptr<Student> stup2 = stup;
    cout << wstup.use_count() << endl;
    shared_ptr<Student> stup3 = stup;
    cout << wstup.use_count() << endl;
    shared_ptr<Student> stup4 = stup;
    cout << wstup.use_count() << endl;
    func(stup4);
    cout << wstup.use_count() << endl;
}

int main(int argc,const char* argv[])
{
```

```
    test_weak();
    cout << wstup.use_count() << endl;
}
```

二十、运算符重载的局限性

1、无法重载的运算符

```
. 成员访问运算符
:: 作用域运算符
sizeof 长度运算符
?: 条件运算符
# 预处理符号
```

2、单目运算符只适合重载为成员函数

单目运算符的运算对象只有一个，只有无法修改该对象的代码时，才适合重载为全局函数，但重载全局函数需要在运算对象的类内设置友元，所以语法上支持，但实际情况是个死局。

3、重载运算符时要注意的问题

1、重载运算符的目标是为了提高代码可读性、实用性，如果达不该效果建议定义为函数。

2、重载运算符函数时，要符合情况情理，不要反人性，不要成为你炫技的工具。

3、重载无法改变运算符的优先级和运算对象个数。

二一、类继承

1、什么是类继承

1、在解决一个问题之前，先考虑现有的类是否能解决部分问题，如果有则继承该类，在此基础上进行扩展，以缩短解决问题的时间，代码复用。

2、在解决一个复杂庞大的问题时，把问题拆分成若干个小问题，每个小问题实现一个类去解决，最后把这若干个类通过继承进行汇总，达到解决问题的目的，这个方式可以降低问题的规模在难度，也方便团队分工、协作。

2、类继承的语法格式

```
class 子名 : <继承方式 父类1>, <继承方式 父类2> // 继承表
{
    成员变量;
public:
    成员函数;
};
```

3、继承方式对访问权限的影响

成员的访问权限：

访问限定符	类内	子类	类外
private	可以	不可以	不可以
public	可以	可以	可以
protected	可以	可以	不可以

```
#include <iostream>
using namespace std;

class Base
{
private:
    int private_num;
protected:
    int protected_num;
public:
    int public_num;
    Base(void)
    {
        private_num = 1;
        protected_num = 2;
        public_num  = 3;
    }

    // 在类内都能访问
    void show(void)
    {
        cout << private_num << endl;
        cout << protected_num << endl;
        cout << public_num << endl;
    }
};

class Test : public Base
{
public:
    void show(void)
    {
        // 在子类中不能直接访问父类的private成员
        cout << private_num << endl;
        cout << protected_num << endl;
        cout << public_num << endl;
    }
};

int main(int argc,const char* argv[])
{
    Test t;
    cout << sizeof(t) << endl;
    t.show();

    /*
    Base b;
```

```
    b.show();
    在类外只能public能访问
    cout << b.private_num << endl;
    cout << b.protected_num << endl;
    cout << b.public_num << endl;
    */
    return 0;
}
```

注意：类成员变量和成员函数一旦设置的访问属性，永远不会改变。

继承方式对成员的影响：

父类中成员访问权限	private继承后在子中类	public继承到子类	protected继承到子类
private	private	private	private
public	private	public	protected
protected	private	protected	protected

1、父类的成员被子类继承后，通过子类对象如果能在类外访问，则说明继承到子类后是public访问属性。

2、父类A的成员被子类B继承后，然后子类B再被孙子类C继承，在孙子类C的内部，如果能访问子类B中的成员，则说明该成员至少是protected，不能访问则说明是private。

3、继承方式只能影响父类成员继承到子类后变成什么访问方式（也就是成员被子类继承后，成员在外界、孙子类的访问权限），并不能影响父类成员在子类中的访问权限(是由父类中设置的访问权限决定的)。

4、C++语言的继承特点

1、C++的继承语法可以继承多个父类，这种语法叫多重继承。

2、子类继承父类会获取到父类中的所有内容，包括私有成员，即使不能直接访问，也会继承。

3、子类中的成员会隐藏父中的同名成员，在子类内部或通过子类对象访问的同名成员时，访问到的是子类的成员，但通过类名限定符在权限允许的情况下可以访问父类的同名成员。

```
#include <iostream>
using namespace std;

class Base
{
public:
    int num;
    Base(void)
    {
        num = 1234;
    }
};

class Test : public Base
{
public:
    int num;
```

```
    Test(void)
    {
        num = 5678;
        cout << num << endl;
        cout << Base::num << endl;
    }
};

int main(int argc,const char* argv[])
{
    Test t;
    cout << t.Base::num << endl;
    return 0;
}
```

4、子类对象（指针、引用）可以转换成父类对象（指针、引用），这是一种缩小转换，父类对象不能转换成子类对象，因为这是一种放大转换，子类对象转换成父类对象后，访问同名成员时，访问的是父类的成员变量和成员函数(因为子类会继承父类中的所有成员)。

```
#include <iostream>
using namespace std;

class Base
{
public:
    void func(void)
    {
        cout << "我是Base的func函数" << endl;
    }
};

class Test:public Base
{
public:
    void func(void)
    {
        cout << "我是Test类的func" << endl;
    }
};

int main(int argc,const char* argv[])
{
    Test* t = new Test;
    t->func();
    t->Base::func();
    Base* b = t;
    b->func();
    return 0;
}
```

5、子类对象可以直接调用父类的成员函数，子类对象计算出的地址，可以赋值给父类成员函数的this指针。

```
int main(int argc,const char* argv[])
{
    Test t;
    Base b = t;
    b = t;

    Base* p = &t;
    Base& bn = t;
    return 0;
}
```

6、当子继承多个父类时，子类会按照继承表的顺序挨个排列父类对象中的成员，当子类对象向父类对象转换时，会根据继承表计算出偏移值。

```
#include <iostream>
using namespace std;

class A
{
    int num_a;
};
class B
{
    int num_a;
};
class C
{
    int num_a;
};

class Test:public A,public B,public C
{

};

int main(int argc,const char* argv[])
{
    Test t;
    cout << &t << endl;
    A* a = &t;
    cout << a << endl;
    B* b = &t;
    cout << b << endl;
    C* c = &t;
    cout << c << endl;
    return 0;
}
```

7、子类的成员函数与父类的成员函数重名构成不是重载关系，因为它们的作用域不同，构成的是隐藏关系(子类会隐藏父类的同名成员)。

```
#include <iostream>
using namespace std;

class Base
{
```

```
public:
    void func(void)
    {
        cout << "我是父类的成员函数" << endl;
    }
};

class Test : public Base
{
public:
    void func(int num)
    {
        cout << "我是子类的成员函数" << endl;
    }
};

int main(int argc,const char* argv[])
{
    Test t;
    t.func(1234);
    // t.func(); 编译会报错
    t.Base::func();
    return 0;
}
```

二二、对象的创建与释放过程

1、创建对象的过程

1、先为对象分配内存

```
Test t; // 可能在stack、data、bss分配内存
Test* p = new Test; // 在heap分配内存
```

2、按照继承表的顺序调用父类无参构造函数，可以在初始化列表显式调用父类的有参构造，但要注意父类构造的执行顺序与初始化列的调用顺序无关。

3、按照成员的定义顺序执行成员的无参构造函数，可以在初始化列表显式调用成员的有参构造。但要注意成员的构造执行顺序与初始化列表的调用顺序无关。

4、在初始化列表中给const成员变量赋值，也可以给普通成员赋值，但没必要。

5、执行对象自己构造函数，在构造函数中为对象的使用做一些准备工作，例如：从文件、数据库加载数据、给指针成员分配堆内存等，由于构造函数没有返回值，尽量不要执行可能失败的事情，因为没有办法反馈给调用者。

6、等构造函数执行完毕后，对象才完成的创建成功。

2、释放对象的过程

> 当对象离开做用域或显式执行了delete/delete[]语句，就开始了释放对象的过程。

1、执行对象自己的析构函数，完毕一些对象消失后的收尾工作，例如：保存数据到文件、数据库，释放指针成员所指向的堆内存。

2、按照成员的定义顺序，逆序调用成员的析构函数。

3、按照继承表的继承顺序，逆序调用父类的析构函数。

4、释放对象自己所占用的内存。

```
#include <iostream>
using namespace std;

class A
{
    int num;
public:
    ~A(void)
    {
        cout << "类A的析构函数" << endl;
    }
    A(int num)
    {
        this->num = num;
        cout << "类A的有参构造" << endl;
    }
};

class B
{
    int num;
public:
    ~B(void)
    {
        cout << "类B的析构函数" << endl;
    }
    B(int num)
    {
        this->num = num;
        cout << "类B的有参构造" << endl;
    }
};

class C
{
    int num;
public:
    ~C(void)
    {
        cout << "类C的析构函数" << endl;
    }
    C(int num)
    {
        this->num = num;
        cout << "类C的有参构造" << endl;
    }
};

class D
{
    int num;
public:
    ~D(void)
    {
        cout << "类D的析构函数" << endl;
```

```
    }
    D(int num)
    {
        this->num = num;
        cout << "类D的有参构造" << endl;
    }
};
class E
{
    int num;
public:
    ~E(void)
    {
        cout << "类E的析构函数" << endl;
    }
    E(int num)
    {
        this->num = num;
        cout << "类E的有参构造" << endl;
    }
};

class F
{
    int num;
public:
    ~F(void)
    {
        cout << "类F的析构函数" << endl;
    }
    F(int num)
    {
        this->num = num;
        cout << "类F的有参构造" << endl;
    }
};

class Test:public A,public B,public C
{
    D d;
    E e;
    F f;
public:
    Test(void):C(3),B(2),A(1),f(6),e(5),d(4)
    {
        cout << "子类的无参构造" << endl;
    }
    ~Test(void)
    {
        cout << "子类的析构函数" << endl;
    }
};

int main(int argc,const char* argv[])
{
    Test* t = new Test;
    delete t;
    return 0;
```

```
}
```

二三、钻石继承与虚继承

1、什么是钻石继承

假定有一个A类，B类继承A类，C类也继承的A类，如果D类同时继承了B类和C类，那么这种继承结构就叫钻石继承，也就是多重继承时，多个父类有一个共同的父类。

2、钻石继承危害

1、每个父类都会继承一份爷爷类中的成员，子类会继承每个父类中的成员，那么多个父类中的爷爷类成员就会在子类汇聚，存在多份，也就是同样的内容存在多份，就造成了冗余，就会形成内存的浪费。

2、如果通过子类对象或者在子类的内部访问爷爷类中的成员，编译器无法分辨使用从B类中继承的还是从C类中继承的，编译就会报错。

3、要想解决这类问题就需要使用一种特殊的继承方式，虚继承。

```
#include <iostream>
using namespace std;

class A
{
    int num;
public:
    ~A(void)
    {
        cout << "类A的析构函数" << endl;
    }
    A(void)
    {

    }
};

class B : public A
{
public:
    ~B(void)
    {
        cout << "类B的析构函数" << endl;
    }
    B(void)
    {
    }
};

class C : public A
{
public:
    ~C(void)
    {
        cout << "类C的析构函数" << endl;
    }
    C(void)
    {
```

```

    }
};

class D : public B,public C
{
public:
    ~D(void)
    {
        cout << "类D的析构函数" << endl;
    }
    D(void) {}
    void show(void)
    {
        cout << num << endl;
    }
};

int main(int argc,const char* argv[])
{
    cout << sizeof(A) << endl;
    cout << sizeof(B) << endl;
    cout << sizeof(C) << endl;
    cout << sizeof(D) << endl;
    return 0;
}
```

3、什么是虚继承

父类在继承爷爷类时，在继承方式的前面增加 **virtual** 关键字就能解决，冗余和调用冲突的问题。

```
#include <iostream>
using namespace std;

class A
{
protected:
    int num;
public:
    ~A(void)
    {
        cout << "类A的析构函数" << endl;
    }
    A(void)
    {

    }
};

class B : virtual public A // 虚继承
{
public:
    ~B(void)
    {
        cout << "类B的析构函数" << endl;
    }
    B(void)
```

```
    {
    }
};

class C : virtual public A // 虚继承
{
public:
    ~C(void)
    {
        cout << "类C的析构函数" << endl;
    }
    C(void)
    {

    }
};

class D : public B,public C
{
public:
    ~D(void)
    {
        cout << "类D的析构函数" << endl;
    }
    D(void) {}
    void show(void)
    {
        cout << num << endl;
    }
};

int main(int argc,const char* argv[])
{
    cout << sizeof(A) << endl;
    cout << sizeof(B) << endl;
    cout << sizeof(C) << endl;
    cout << sizeof(D) << endl;
    return 0;
}
```

4、虚继承的原理

1、父类继承爷爷类时使用了虚继承，父类中就多4个字节，这4个字节就个指针（虚指针），该指针指向一个表格，表格中记录了从爷爷类对象中继承的内容。

2、孙子类会继承所有父类中的虚指针，然后对多个表格中的内容进行对比，如果从父类继承的成员是爷爷类中的，那么这种成员在孙子类中只保留一份。

3、虚继承没有太大的实用价值，因为其它一些面向对象的编程语言，如：Java只允许继承一个父类，就说明单继承可以解决所有问题，从而规避钻石继承带来的问题，因此我们在编写C++代码时，尽量只继承一个父类。

4、但要小心笔试题中的虚继承。

二四、虚函数与函数覆盖(重写)

1、什么虚函数

使用 virtual 关键字修饰的成员函数，被称为虚函数。

2、什么是函数覆盖

如果父类中有虚函数，且子类中有与父类的虚函数**函数签名相同**成员函数，此时子类就会把从父类继承的虚函数覆盖掉。

当使用父类指针或引用，指向子类对象时，再通过父类对象的指针或引用调用虚函数时，访问到的是子类的同名函数。

也就是父类指针或引用，实际指向的是哪种对象，调用的就那个对象有成员函数，这种现象建立在函数覆盖的前提下。

3、构成函数覆盖的条件

1、一定发生在父子类之间。

2、父类中的成员函数是虚函数。

3、子类中的成员函数与父类虚函数的函数签名相同。

函数签名相同：

1、参数列表完全相同(参数的个数，类型完全相同，如果有指针类型的参数，const属性也要相同，函数的常属性也要相同，异常声明也要相同)。

2、返回值类型完全相同或是具有继承关系的父子类(如果参数列表已经相同，但返回值类型不同，编译会报错)。

4、函数覆盖的原理

1、一旦类中定义的虚函数，该类就会多一个隐藏指针成员（虚函数表指针），该指针指向一个表格，该表格中记录了父类中所有虚函数的地址。

```
int main(int argc,const char* argv[])
{
    Base b;
    cout << sizeof(b) << endl; // 结果是4，因为该空类中有一个虚函数表指针

    //((A* (*)(int*))(*(int*)(*(int*)&b)))(NULL);
    void* p = (void*)*(int*)&b;
    void* fp = (void*)*(int*)p;
    ((A* (*)(int*))fp)(NULL);
}
```

2、子类会继承父类中的虚函数表指针，编译器会检查子类中有没与虚函数表中函数签名相同的成员函数，如果有则会把子类成员函数地址替换到虚函数表中。

```
int main(int argc,const char* argv[])
{
    Test t;
    cout << sizeof(t) << endl;
    //((A* (*)(int*))(*(int*)(*(int*)&b)))(NULL);
    void* p = (void*)*(int*)&t;
    void* fp = (void*)*(int*)p;
    ((A* (*)(int*))fp)(NULL);
}
```

3、当父类的指针或引用指向一个子类对象时，然后再调用虚函数，编译器就会按照虚函数表中的函数地址进行调用，如果实际对角是子类对象，调用的就是覆盖后的子类成员函数。

```
#include <iostream>
using namespace std;

class A
{
public:
    void test(void)
    {

    }
};

class B : public A
{
public:
    void test(void)
    {
    }
};

class Base
{
public:
    virtual A* func(int* p)
    {
        cout << "我是Base的func函数" << endl;
    }
};

class Test:public Base
{
public:
    B* func(int* p)
    {
        cout << "我是Test类的func" << endl;
    }
};

int main(int argc,const char* argv[])
{
        /*
    Test* t = new Test;
    t->func();
```

```
    t->Base::func();
    Base* b = t;
    b->func();
    Test t;
    Base b = t;
    b.func(NULL);

    Base* p1 = &t;
    p1->func(NULL);
    Base& p2 = t;
    p2.func(NULL);
    */
    /*
    Test t;
    cout << sizeof(t) << endl;
    //((A* (*)(int*))(*(int*)(*(int*)&b)))(NULL);
    void* p = (void*)*(int*)&t;
    void* fp = (void*)*(int*)p;
    ((A* (*)(int*))fp)(NULL);
    */

    Test t;
    Base& b = t;
    b.func(NULL);
    Base* p = &t;
    p->func(NULL);
}
```

二五、多态

1、什么是多态

指令或语句的多种形态，也就指令或语句会根据环境的不同、参数不同会产生多种执行结果，这就叫多态。

2、编译时多态（静态多态）

指令或语句在编译时，编译器能根据环境或参数确定执行结果，这就叫作编译时多态，也叫静态多态，例如：调用重载过的函数。

```
#include <iostream>
using namespace std;

void func(int num)
{
    cout << "func(int num)" << endl;
}

void func(short num)
{
    cout << "func(short num)" << endl;
}

int main(int argc,const char* argv[])
{
    //short num = 1234;
    int num = 1234;
```

```

    func(num);
    return 0;
}
```

3、运行时多态（动态多态、类多态）

当定义一个父类的引用或指针时，使用它调用虚函数，它可能调用的是父类中的虚函数，也可能调用子类中的成员函数，但具体调用哪个版本的函数，要在程序运行时根据实际类对象才能决定。

```
#include <iostream>
using namespace std;

class Base
{
public:
    virtual void func(int* p)
    {
        cout << "我是Base的func函数" << endl;
    }
};

class Test:public Base
{
public:
    void func(int* p)
    {
        cout << "我是Test类的func" << endl;
    }
};

int main(int argc,const char* argv[])
{
    Base* p;
    char cmd;
    cout << "你是0还是1" << endl;
    cin >> cmd;
    if('0' == cmd)
        p = new Test;
    else
        p = new Base;
    p->func(NULL);

    return 0;
}
```

4、构成类多态的原理

当使用父类的指针或引用指向一个对象时，然后通过父类指针或引用去调用虚函数，此时它会从虚函数指针所指向的虚函数表中寻找函数签名相同的函数调用。

如果指针或引用实际指向的是父类对象，那么虚函数表中记录就是父类中的虚函数地址，如果指针或引用实际指向的是子类对象，那么虚函数表中记录的就是覆盖后子类成员函数。

函数覆盖就是类多态的基础。

5、类多态的优点

1、使用类多态可以让调用者只关注调用指令，不需要关注背后是如何实现的。

2、作为多态的实现者，多态提高的代码的安全性、封装性，并且当代码需要更新和升级时，不需要通知调用者。

二六、纯虚函数和抽象类

1、什么是纯虚函数

在虚函数(类成员函数)声明的末尾添加 **=0** 这种函数就叫纯虚函数，这种函数不需要实现，这种函数生来就被覆盖的。

2、什么是抽象类

类中有纯虚函数，这种类就叫抽象类，这种类不能创建类对象，必须被继承，不然就没有意义。

抽象类可以定义指针或引用，去指向子类对象，然后再根据实际指向的对象调用对象内的成员函数（有多个子类）。

子类一旦继承了抽象类，必须把它的所有纯虚函数全部覆盖，否则子类也就变成了抽象类，不能创建对象。

3、什么是纯抽象类

所以有成员函数都是纯虚函数，这种类就叫纯抽象类，是在C++语言中一种特殊的接口类，不负责实现具体的功能代码，它的意义在于告诉使用者过如何调用相关的接口，并且也限制继承者被调用时能得到哪些参数。

```
#include <iostream>
#include <stdlib.h>
#include <time.h>
using namespace std;

class Base
{
public:
    // 纯虚函数
    virtual void func(void) = 0;
    void test(void)
    {
        cout << "我base的成员函数" << endl;
    }
};
class A : public Base
{
public:
    void func(void)
    {
        cout << "我是A类的func函数" << endl;
    }
};
class B : public Base
{
public:
    void func(void)
    {
```

```
        cout << "我是B类的func函数" << endl;
    }
};
class C : public Base
{
public:
    void func(void)
    {
        cout << "我是C类的func函数" << endl;
    }
};

class D : public Base
{
public:
    void func(void)
    {
        cout << "我是D类的func函数" << endl;
    }
};

Base* getObject(int cmd)
{
    switch(cmd)
    {
    case 0: return new A;
    case 1: return new B;
    case 2: return new C;
    case 3: return new D;
    }
}

int main(int argc,const char* argv[])
{
    srand(time(NULL));

    Base* b = getObject(3);
    b->func();
    b->test();
    return 0;
}
```

二七、虚构造与虚析构

1、构造函数能否是虚函数，为什么?

对象有创建过程:

1、给对象分配内存

2、根据继承表顺序调用父类构造

3、根据成员对象的的定义顺序调用成员对象的构造函数

4、执行对象自己的构造函数

如果父类的构造函数函数设计成虚函数并且被子类覆盖（如果虚函数没有被覆盖就设计的没有意义），当创建子类对象时，先调用父类的虚构造，但此时实际对象是子类对象，根据多态的特性此时会转而执行子类的构造（调用虚函数表中覆盖后的版本），但执行子类构造函数前需要先执行父类构造，这样就形成了死循环，所以构造函数不能设计成虚函数。

2、析构函数能否是虚函数，为什么?

对象的释放过程：

1、执行对象自己的析构函数

2、根据成员对象的创建过程逆序执行成员对象的析构函数

3、根据继承表的顺序逆序执行父类的析构函数

4、释放对象的内存

假如父类的析构函数设计成虚函数并且被子类覆盖，当释放子类对象时，先执行子类对象的析构函数，然后执行父类对象的析构函数，此时子类对象已经被释放完毕，所以无法形成多态，只会执行父类的析构函数，不会产生任何错误，所以析构函数可以是虚函数。

```
#include <iostream>
using namespace std;

class Base
{
public:
    virtual ~Base(void)
    {
        cout << "Base类的析构函数" << endl;
    }
};

class Test:public Base
{
public:
    ~Test(void)
    {
        cout << "Test类的析构函数" << endl;
    }
};

int main(int argc,const char* argv[])
{
    Test t;
}
```

3、什么进修需要设计虚析构

当使用类多态时，使用父类指针、引用去释放子类对象时，如果析构函数没有设计成虚函数(没有覆盖)，那么将只执行父类的析构函数（无法调用子类的析构函数），如果子类中有指针成员且指向堆内存，这种情况下就会造成内存泄漏。

注意：当使用类多态时，且子类成员中有指针指向堆内存，必须要把父类的析构函数设计成虚函数。

```
#include <iostream>
using namespace std;

```

```
class Base
{
public:
    virtual void func(void)
    {
        cout << "Base类的func函数" << endl;
    }
    virtual ~Base(void)
    {
        cout << "Base类的析构函数" << endl;
    }
};

class Test:public Base
{
    int* p;
public:
    Test(void)
    {
        p = new int;
        cout << "new " << p << endl;
    }
    void func(void)
    {
        cout << "Test类的析构函数" << endl;
    }
    ~Test(void)
    {
        delete p;
        cout << "Test类的析构函数 delete " << p << endl;
    }
};

int main(int argc,const char* argv[])
{
    Base* p = new Test;
    p->func();
    delete p;
}
```

二八、类型信息运算符

1、什么类型信息运算符

C++中有这个typeid关键字，用于获取数据的类型信息。

2、类型信息运算符的作用。

当我们使用类多态时，我们很难通过肉眼识别出对象的真实类型，如果父子类形成了多态，使用typeid就可以获取到对象的真实类型。

以及判断是否函数，还是函数指针，判断标识符是否指针变量、是否是二级指针。

3、使用方法

1、需要包含头文件 #include

2、typeid(数据) 会返回一个记录数据类型信息的类对象(type_info)

3、type_info 有一个name成员函数，会以字符串形式返回类型的名字：

1、基本类型返回类型的缩写

2、指针类型以P开头

3、带const属性的，名字中会带K

4、复合类型的会返回长度+名字

5、如果父类引用指向了子类对象，只要父类中定义的虚函数，typeid就可以识别出真实的对象类型。

6、如果父类指针指向了子类对象，父类中定义的虚函数，typeid(*指针)才可以识别出真实的对象类型。

4、type_info 有一些成员函数和运算符函数：

判断一个标识符是否是指针变量，__is_pointer_p()
判断一个标识符是否是函数， __is_function_p()

判断两种类型是否是同一种，==，!=

```
#include <typeinfo>
using namespace std;

struct Student
{

};

class Base
{
public:
    virtual void func(void)
    {

    }
};

class Test : public Base
{
};

int main(int argc,const char* argv[])
{
    cout << typeid(char).name() << endl;
    cout << typeid(short).name() << endl;
    cout << typeid(int).name() << endl;

    Student s;
    cout << typeid(s).name() << endl;

    Test t;
```

```
    Base& b = t;
    cout << typeid(b).name() << endl;

    Base* p = new Test;
    cout << typeid(p).name() << endl;
    cout << typeid(*p).name() << endl;

    cout << typeid(main).__is_function_p() << endl;
    cout << typeid(p).__is_pointer_p() << endl;

    cout << (typeid(t) == typeid(b)) << endl;
    cout << (typeid(t) != typeid(b)) << endl;

    return 0;
}
```

二九、强制类型转换

C++语言为了兼容C语言，依然保留着C语言中的强制类型转换语法，但C语言中的强制类型转换有以下缺点：

1、任何类型之间都可以强制类型转换，所以使用起来比较随意，代码的阅读性性差。

2、不会对原数据和目标类型检查，程序员需要对转换的结果负责（可能会出现数据丢失、段错误等问题）。

基于以上原因C++之父在C++中提供一套更安全的强制类型转换，并且C++之父认为好的代码设计不应该会使用到强制类型转换，当程序需要使用强制类型转换时，就说明你的代码设计有问题，程序不应该使用强制类型转换而是应该重新修改代码的设计，所以强制类型转换的语法设计的难以记忆。

1、去常类型转换 const_cast<目标类型>(源数据)

源和目标类型之间除了const属性不同，其它没有任何区别，否则就会产生编译错误，一般用于去掉指针或引用的常属性。

```
#include <iostream>
using namespace std;

int main(int argc,const char* argv[])
{
    const int num = 1234;
    const int* p = &num;
    cout << const_cast<int*>(p) << endl;
    // cout << const_cast<char*>(p) << endl;
    int& num1 = const_cast<int&>(num);
    long num2 = const_cast<long>(num);

    return 0;
}
```

2、静态类型转换 static_cast<目标类型>(源数据)

源数据和目标类型之间必须有一个方向能自动类型转换，否则就会产生编译错误。

```
#include <iostream>
using namespace std;

int main(int argc,const char* argv[])
{
    int num = 1234;
    char ch = static_cast<char>(num);

    void* p = &num;
    int* p1 = static_cast<int*>(p);

    // char* 和int*不具备隐匿转换的功能
    static_cast<char*>(&num);
    float f = 3;
    static_cast<int>(f);

    // 与const相关的应该使用const_cast类型转换
    const int* cp;
    static_cast<int*>(cp);

    return 0;
}
```

3、重解释类型转换 reinterpret_cast<目标类型>(数据)

专用于指针变量的类型转换，主要用于指针与指针的转换，指针与整数的转换，与其它的强制类型转换相比，它的自由度比较高，但也比较危险。

```
#include <iostream>
using namespace std;

class A { };

class B { };

int main(int argc,const char* argv[])
{
    int* p;
    char* p1 = reinterpret_cast<char*>(p);

    int num = reinterpret_cast<int>(p);
    A* a;
    B* b = reinterpret_cast<B*>(a);

    return 0;
}
```

4、动态类型转换 dynamic_cast<目标类型> (数据)

把父类的指针或引用转换成子类的指针或引用，并且父类中必须有虚函数表指针。

```
#include <iostream>
using namespace std;

class A
{
public:
    virtual void func(void)
    {

    }
};

class B : public A
{

};

int main(int argc,const char* argv[])
{
    /*
    int* p;
    dynamic_cast<long*>(p);
    */

    A* a = new A;
    B* b = dynamic_cast<B*>(a);
    return 0;
}
```

三十、C++的文件IO

在C++中把文件的读写操作都封装在标准库中，ifstream类主要用于读取文件内容，ofstream主要用于写入文件内容，fstream类可读可写。

1、打开文件操作

使用构造函数打开文件：

```
fstream( const char *filename, openmode mode);
功能：创建操作文件的类对象，并顺便打开文件
filename：文件的路径
mode：打开的方式或权限
    默认参数是：O_RDWR
    ios::app 添加输出，O_WRONLY|O_CREAT|O_APPEND
    ios::in 为读取打开文件 O_RDONLY
    ios::out 为写入打开文件 O_WRONLY|O_CREAT|O_TRUNC
    ios::binary 以二进制模式打开文件，相当于C语言中fopen函数的带b的打开方式
    ios::ate 当已打开时寻找到EOF，打开文件后顺序设置文件位置指针
    ios::trunc 文件存在则清空文件 O_TRUNC

ifstream( const char *filename, openmode mode);
mode
    默认参数是：O_RDONLY
```

```

ofstream( const char *filename, openmode mode);
mode
    默认参数是：O_WRONLY|O_CREAT|O_TRUNC
```

使用open成员函数打开文件：

```
void open( const char *filename);
void open( const char *filename, openmode mode);
功能：与构造函数参数相同
```

如何判断文件打开成功或失败：

方法1：直接使用类对象进行逻辑判断，因它们重载逻辑运算符。

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc,const char* argv[])
{
    ifstream fs("test1234.txt");
    if(!fs)
    {
        cout << "打开失败" << endl;
        return 0;
    }

    return 0;
}
```

方法2：调用good函数，该函数用于判断对象的上一次操作是否成功，所以也可判断文件打开是否成功。

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc,const char* argv[])
{
    ifstream fs("test.txt");
    if(!fs.good())
    {
        cout << "打开失败" << endl;
        return 0;
    }

    return 0;
}
```

2、文本格式读写

注意：像使用cout一样写入数据，使用cin一样读取数据。

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc,const char* argv[])
{
    fstream fs("test.txt");
    if(!fs.good())
    {
        cout << "打开失败" << endl;
        return 0;
    }

    //fs << "我就试试能不能写成功 " << 1234 << endl;
    string str;
    int num;
    fs >> str >> num;
    printf("%s %d\n",str.c_str(),num);

    return 0;
}
```

注意：如果需要对一个结构、类进行文本格式读写，最好给它重载输入、输出运算符，不光cin、cout可以使用，ofstream、ifsteam也可以使用。

```
#ifndef STUDENT_H
#define STUDENT_H
#include <iostream>
using namespace std;

struct Student
{
    int id;
    string name;
    char sex;
    short age;
    float score;

    friend ostream& operator<<(ostream& os,const Student& stu)
    {
        return os << stu.id << " " << stu.name << " " << stu.sex << " " << stu.age << " " << stu.score << endl;
    }
    friend istream& operator>>(istream& is,Student& stu)
    {
        return is >> stu.id >> stu.name >> stu.sex >> stu.age >> stu.score;
    }
};
#endif//STUDENT_H
```

```
#include <fstream>
#include "student.h"
```

```

int main(int argc,const char* argv[])
{
    ifstream ifs("stu.txt");
    if(!ifs)
    {
        cout << "打开文件失败！" << endl;
        return 0;
    }

    Student stu = {};
    ifs >> stu;
    cout << stu.id << " " << stu.name << " " << stu.sex << " " << stu.age << " " << stu.score << endl;
    return 0;
}
```

```
#include <fstream>
#include "student.h"

int main(int argc,const char* argv[])
{
    ofstream ofs("stu.txt");
    if(!ofs)
    {
        cout << "打开文件失败！" << endl;
        return 0;
    }

    Student stu = {10010,"xixi",'w',22,547};

    ofs << stu;

    return 0;
}
```

3、二进入格式读写

注意：如果是在Windows系统下读写二进制文件，mode参数中要有ios::binary。

```
istream &read( char *buffer, streamsize num );
功能：读取一块数据到内存
buffer：一般情况下需要强制类型转换，特别是结构对象或类对象。
注意：返回值与标准C和Linux系统读取函数不同，需要调用gcount函数获取读取了多少个字节数据。

ostream &write( const char *buffer, streamsize num);
功能：把一块内存中的数据写入文件
注意：返回值与标准C和Linux系统读取函数不同，需要调用good函数判断写入是否成功。
```

注意：如果结构、类成员中有指针成员或string类的成员变量，不能以二进制格式直接把对象保存到文件中，最好以文本格式保存。

```
#include <iostream>
#include <fstream>
using namespace std;
struct Student
```

```
{
    int id;
    string name;
    char sex;
    short age;
    float score;
};

int main(int argc,const char* argv[])
{
    ofstream ofs("stu.bin");

    if(!ofs)
    {
        cout << "文件打开失败！" << endl;
        return 0;
    }

    Student stu = {10010,"hehe",'w',18,650};
    ofs.write((char*)&stu,sizeof(stu));
    cout << ofs.good() << endl;
}
```

```
#include <iostream>
#include <fstream>
using namespace std;
struct Student
{
    int id;
    string name;
    char sex;
    short age;
    float score;
};

int main(int argc,const char* argv[])
{
    ifstream ifs("stu.bin");
    Student* stup = new Student;
    ifs.read((char*)stup,sizeof(Student));
    cout << ifs.gcount() << " " << sizeof(Student) << endl;
    printf("%d %s %c %hd %g\n",stup->id,stup->name.c_str(),stup->sex,stup->age,stup->score);

    return 0;
}
```

4、随机读写

```
istream &seekg( off_type offset, ios::seekdir origin );
ostream &seekp( off_type offset, ios::seekdir origin );
功能：以偏移值+基础位置设置文件的位置指针，之所以这样设计是为了兼容那些有两个文件位置(读写分开)操作系统，使用方法与lseek、fseek类型。
ios::seekdir origin
    ios::beg SEEK_SET
    ios::cur SEEK_CUR
```

```
    ios::end SEEK_END


istream &seekg( pos_type position );
ostream &seekp( pos_type position );
功能：以绝对位置设计文件的位置指针
pos_type：
    把文件的位置指针移动到文件的第几个字节。

pos_type tellg();
pos_type tellp();
功能：获取文件的位置指针，与ftell函数的功能相同。

```

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc,const char* argv[])
{
    fstream fs("test.txt");

    //fs.seekg(3,ios::beg);
    fs.seekg(6);
    char ch;
    fs >> ch;
    cout << ch;
    cout << fs.tellg() << endl;
    //fs.seekg(0,ios::beg);
    fs.seekg(0);

    fs >> ch;
    cout << ch;

    cout << fs.tellg() << endl;

    return 0;
}
```

5、特殊格式的读写

```
fmtflags flags();
fmtflags flags( fmtflags f );
功能：获取当前流的格式标志

fmtflags setf( fmtflags flags );
fmtflags setf( fmtflags flags, fmtflags needed );
功能：设置当前流的格式化标志为flags

void unsetf( fmtflags flags );
清除与当前流相关的给定的标志flags
```

操作符	描述	输入	输出
boolalpha	启用boolalpha标志	X	X
dec	启用dec标志	X	X
endl	输出换行标示，并清空缓冲区		X
ends	输出空字符		X
fixed	启用fixed标志		X
flush	清空流		X
hex	启用 hex 标志	X	X
internal	启用 internal 标志		X
left	启用 left 标志		X
noboolalpha	关闭boolalpha 标志	X	X
noshowbase	关闭showbase 标志		X
noshowpoint	关闭showpoint 标志		X
noshowpos	关闭showpos 标志		X
noskipws	关闭skipws 标志	X	
nounitbuf	关闭unitbuf 标志		X
nouppercase	关闭uppercase 标志		X
oct	启用 oct 标志	X	X
right	启用 right 标志		X
scientific	启用 scientific 标志		X
showbase	启用 showbase 标志		X
showpoint	启用 showpoint 标志		X
showpos	启用 showpos 标志		X
skipws	启用 skipws 标志	X	
unitbuf	启用 unitbuf 标志		X
uppercase	启用 uppercase 标志		X
ws	跳过所有前导空白字符	X	

```
#include <iostream>
using namespace std;

int main(int argc,const char* argv[])
{
    /*
    printf("|%4d|\n",1);
```

```
    cout << "|";
    cout.width(4);
    cout << 1 << "|" <<  endl;
    */
    int num = 0x01020304;
    printf("%x\n",num);
    cout << hex << num << endl;
    bool flags = false;
    cout << boolalpha << flags << endl;
    return 0;
}
```

三一、 异常处理

1、什么是异常处理

从宏观角度来说，异常处理就是当程序执行过程中出现了错误，以及对错误的处理方案。

2、C语言的异常处理

C语言一般通过函数返回值、信号，来表示程序在运行过程中出现的错误。

文件打开失败，fopen、open函数的返回值来判断文件打开是否成功。

缺点：返回的类型单一，返回的数据很难跨作用域，还需要考虑它们的成功情况，必须使用if、switch对返回值进行判断。

断错误、非法硬件指令、总线错误、浮点异常等代码执行过程中出现错误信息。

缺点：错误信息过于简单，捕获处理完后进行依然需要结束。

3、C++语言的异常处理

如何抛异常：

throw 数据;

类似return语句返回一个数据，但不同是它可以返回任何类型的数据，并且可以不需要预告声明。

声明异常：

1、所谓的异常声明，就是函数的实现者对调用者的一种承诺，我会抛哪些类型的异常。

```
返回值类型 函数名(参数列表) throw(类型,...)
{

}
```

2、如果函数不进行异常声明，则表示可能会抛出任何类型的异常。

3、如果抛出了声明以外的异常，编译不会出错，但无法捕获，即使你写的准确捕获语句，也无法捕获，也就是说如果函数的实现者不遵守承诺，调用它的程序只有死路一条。

4、throw() 表示不会抛出任何异常，请放心调用。

```
size_t file_size(const char* filename)
{
    throw 1234;
    ifstream ifs(filename);
```

```
    if(!ifs)
    {
        throw string("文件打开失败！");
    }

    ifs.seekg(0,ios::end);
    return ifs.tellg();
}

int main(int argc,const char* argv[])
{
    try{
        cout << file_size("eheheh
```

捕获异常：

```
try{
    可能产生异常的函数调用、代码。
}
catch(类型1 变量名){
    1、处理异常
    2、继续往上抛
}
catch(类型2 变量名){
    1、处理异常
    2、继续往上抛
}
...
```

```
int main(int argc,const char* argv[])
{
    int* p;
    try{
        p = new int[0xffffffff];
    }
    catch(bad_array_new_length error){
        cout << "申请内在失败" << endl;
        cout << error.what() << endl;
    }
}
```

注意：如果继续往上抛的异常，最终异常没有被处理，那么程序将停止执行。

抛异常和捕获异常要注意的问题：

1、在捕获异常时catch不会挑选最合适的，而从上到下选择一个可以捕获的类型，所应该先写捕获子类异常的语句，再写捕获父类异常的语句，或者只写捕获父类异常变量，这样返回父类异常和子类异常都可以兼容。

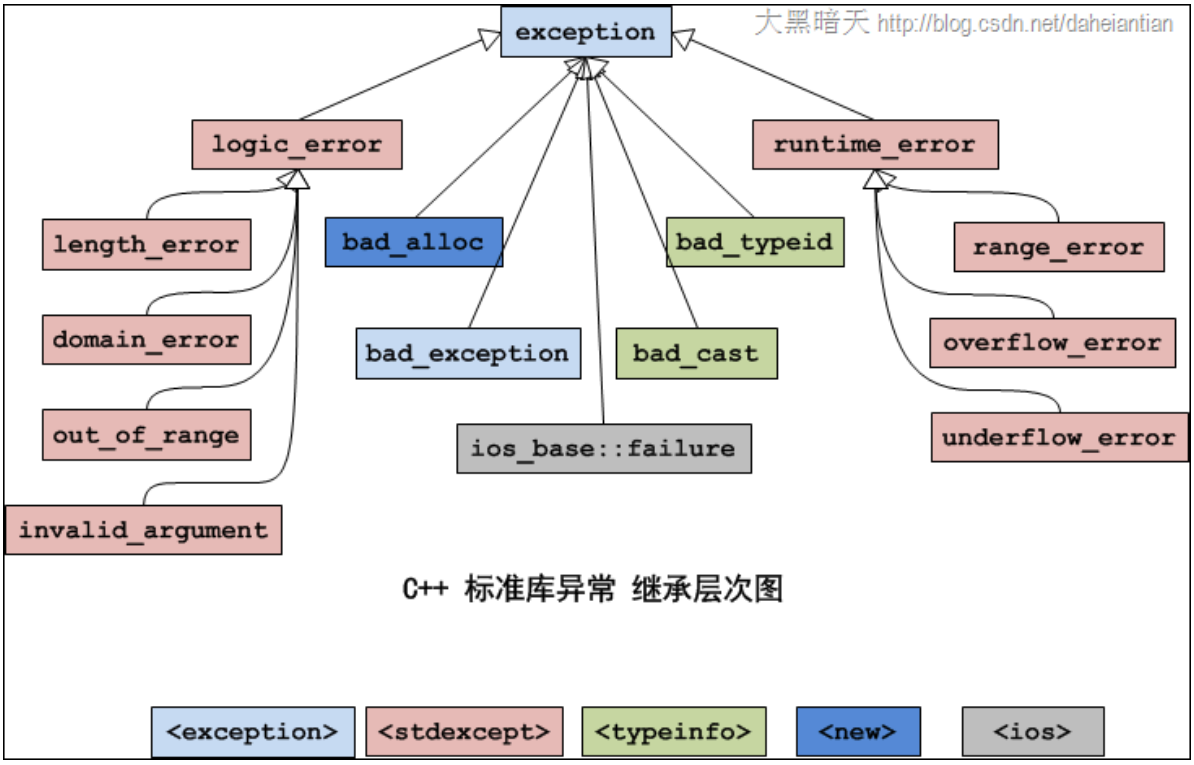
2、不要在析构函数的析构函数中抛出异常，如果该类对象就是异常数据，那么会在抛异常的过程中产生新的异常。

3、不要抛指针类型的异常，因为我们的异常是跨作用域的，当捕获者获得异常后，指针指向的内存可能已经释放，那么捕获的指针就可能是野指针。

4、尽量使用类名创建临时的类对象进行抛异常，并且使用引用来捕获异常，因为这样可以避免调用拷贝构造函数，也避免对象出了作用域后被释放产生悬空引用。

4、C++标准异常

所谓的C++标准异常就是在使用C++标准库中的函数、类、类对象、new、delete时可能抛出的异常，简称C++标准异常。



异常	描述
std::exception	该异常是所有标准 C++ 异常的父类。
std::bad_alloc	该异常可以通过 **new** 抛出。
std::bad_cast	该异常可以通过 **dynamic_cast** 抛出。
std::bad_exception	这在处理 C++ 程序中无法预期的异常时非常有用。
std::bad_typeid	该异常可以通过 **typeid** 抛出。
std::logic_error	理论上可以通过读取代码来检测到的异常。
std::domain_error	当使用了一个无效的数学域时，会抛出该异常。
std::invalid_argument	当使用了无效的参数时，会抛出该异常。
std::length_error	当创建了太长的 std::string 时，会抛出该异常。
std::out_of_range	该异常可以通过方法抛出，例如 std::vector 和 std::bitset<>::operator。
std::runtime_error	理论上不可以通过读取代码来检测到的异常。
std::overflow_error	当发生数学上溢时，会抛出该异常。
std::range_error	当尝试存储超出范围的值时，会抛出该异常。
std::underflow_error	当发生数学下溢时，会抛出该异常。

```
#include <exception>
using namespace std;

int main(int argc,const char* argv[])
{
    // 只要是C++标准异常，该方法都可以捕获
```

```
    try{
        // int* p = new int[0xffffffff];
        // string str(0xffffffff,'x');
        int a = 1234 , b = 0;
        int c = a / b;
    }
    catch (exception& ex)
    {
        cout << ex.what() << endl;
    }

    return 0;
}
```

5、自定义的通用异常

```
#ifndef MY_EXCEPTION_H
#define MY_EXCEPTION_H
#include <iostream>
#include <sstream>
#include <exception>
using namespace std;

class MyException : public exception
{
    string error;
public:
    MyException(const string& info,const string& file,const string& func,size_t line)
    {
        error = info;
        error += " ";

        time_t time = ::time(NULL);
        stringstream ss;
        ss << file << " " << func << " " << line << " " << ctime(&time);
        error += ss.str();
    }
    const char* what(void) const throw()
    {
        return error.c_str();
    }
    ~MyException(void) throw()
    {

    }
};

#define MY_EXCEPTION(info) MyException(info,__FILE__,__func__,__LINE__)

#endif//MY_EXCEPTION_H
```

```
#include <iostream>
#include <fstream>
#include "my_exception.h"

```

```
using namespace std;

size_t file_size(const string& filename)
{
    ifstream ifs(filename.c_str());
    if(!ifs)
    {
        throw MY_EXCEPTION("文件存在");
    }

    ifs.seekg(0,ios::end);
    return ifs.tellg();
}

int main(int argc,const char* argv[])
{
    try{
        file_size("heheheh");
    }
    catch(exception& ex)
    {
        cout << ex.what() << endl;
    }
    return 0;
}
```